DOCUMENTO CONSOLIDADO DEL PROYECTO · VERSIÓN 2.0

RutaVida

Sistema Inteligente de Rutas de Emergencia para Valledupar

Planteamiento del problema, requerimientos, casos de uso, arquitectura de software, diagrama de clases, base de datos, motor de rutas, estado de la implementación y organización del equipo.

Proyecto	Asignatura	Institución
RutaVida — Sistema de Rutas de Emergencia	Programación de Computadores III + Base de Datos	Universidad Popular del Cesar · Ingeniería de Sistemas

Integrante	Rol en el proyecto
Andrés	Líder del proyecto · Base de datos, DAO e integración
Kendrick	Service y motor de rutas
Diego	Vista (interfaz) y versión móvil
Sherly	Model, Controller, Util y pruebas

Valledupar, 14 de octubre de 2026

> **Aviso**
>
> RutaVida es un **prototipo académico**. No reemplaza un servicio oficial de despacho de emergencias y no debe usarse para coordinar emergencias reales sin validación, autorización e infraestructura apropiadas. Los datos de tráfico y de posición de la primera versión son **simulados** y así se identifican en el sistema.

Control de versiones

Versión	Fecha	Cambios
1.0	13/10/2026	Documento consolidado inicial: planteamiento, propuesta ampliada, diseño de base de datos, diseño técnico del motor de rutas y organización del equipo.
2.0	14/10/2026	Base de datos en Oracle XE local; catálogo único de estados; diccionario de datos corregido y **ejecutado y verificado**; casos de uso; arquitectura implementada con GRASP y SOLID; diagrama de clases; estado de la implementación y pruebas; plan del Sprint 1.

Cambios importantes respecto a la versión 1.0

Tema	Versión 1.0	Versión 2.0	Motivo
Base de datos	Oracle alojado en la nube (Oracle Cloud)	**Oracle Database XE 18c local** (`localhost:1521/XEPDB1`), una por integrante	Decisión del equipo (14/10). Pendiente informar al profesor. El código no cambia si se vuelve a la nube: solo la URL de conexión
Vista	Swing en la propuesta y JavaScript en el plan	**Pendiente**: el profesor decide entre Swing y web	No se implementan las dos
Framework	Spring Boot	**Sin framework**: inyección de dependencias manual en `AppContext`	Menos tecnología nueva y más fácil de explicar
Tiempo real	Contradicción entre documentos	**Núcleo obligatorio, con datos simulados** identificados como `SIMULACION`	Unificado con la propuesta de tiempo real
Estados	Mezcla de mayúsculas, minúsculas y tildes	**Catálogo único** en mayúsculas, sin tildes, con _	Evitar errores entre Java y Oracle
TRAMO_VIAL	Una fila para los dos sentidos	**Una fila por sentido** de circulación	El grafo es dirigido
Incidentes	Activo / cerrado	**PENDIENTE** por defecto; solo `ACTIVO` afecta las rutas	Un reporte falso nunca cierra una calle
Driver JDBC	—	`ojdbc11` 21.9.0.0	El driver 23 solo es compatible con Oracle 19c o posterior

Contenido

*Si el índice aparece vacío, en Word: clic derecho sobre él → *Actualizar campos* → *Actualizar toda la tabla*.*

1. Planteamiento del problema

1.1 Contexto

En una emergencia (accidente de tránsito, incendio, urgencia médica o rescate), el tiempo que tarda una ambulancia o un vehículo de bomberos en llegar al sitio, y cuando aplica en trasladar al paciente a un hospital, depende en gran medida de la ruta elegida. Elegirla de forma manual, sin considerar el estado actual de las vías, es lento y propenso a error.

En Valledupar el estado de las vías cambia a lo largo del día: la congestión, los semáforos, los accidentes, las obras y los cierres modifican el tiempo real que toma recorrer cada tramo. Por eso **la ruta más corta en distancia no siempre es la más rápida**.

1.2 Problema a resolver

> **Pregunta central**
>
> ¿Cómo calcular la ruta más rápida para una ambulancia o un vehículo de bomberos en Valledupar, usando información de tráfico que cambia en el tiempo, y cómo decidir qué unidad se asigna a cada emergencia?

1.3 Propuesta de solución

RutaVida es un sistema de gestión y optimización de rutas de emergencia. Registra emergencias, localiza las unidades disponibles, calcula una ruta recomendada y dos alternativas según el estado de las vías, explica por qué una ruta es preferible, sigue el desplazamiento de la unidad y recalcula la ruta cuando cambian las condiciones.

- **Modelo de grafo:** las intersecciones son nodos y los tramos de vía son aristas dirigidas.
- **Algoritmo de caminos mínimos:** A* (y Dijkstra para comparar) con pesos dinámicos medidos en segundos: distancia ÷ velocidad, más penalizaciones por congestión, semáforos e incidentes.
- **Asignación de unidades:** problema de Investigación de Operaciones; con varias emergencias simultáneas se resuelve con el algoritmo húngaro.
- **Tiempo real:** un simulador cambia el tráfico y mueve las unidades; el sistema detecta cuándo una ruta activa se ve afectada y la recalcula.
- **Información centralizada:** emergencias, unidades, hospitales, red vial, rutas y auditoría en una base de datos Oracle.

1.4 Alcance por fases

Fase	Descripción	Estado
Fase 1 — Simulación	Red vial real de Valledupar (extracto acotado de OpenStreetMap) en la base de datos; tráfico, semáforos, incidentes y posiciones de las unidades generados por un simulador. Es la fase que se entrega y se sustenta.	En desarrollo
Fase 2 — Datos reales	Tráfico de una fuente externa, posiciones GPS de las unidades, autenticación reforzada y despliegue. Se presenta como arquitectura preparada: se cambia la fuente de datos sin modificar el resto del sistema.	Arquitectura preparada

Lo que el sistema no hace

No controla semáforos reales ni accede a sus controladores.

No recibe datos reales de GPS ni de tráfico en la fase 1: todo dato simulado se marca como `SIMULACION`.

No reemplaza un servicio oficial de despacho.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar RutaVida, un sistema desarrollado en Java con arquitectura en capas y patrón DAO, respaldado por una base de datos relacional Oracle, capaz de gestionar emergencias, unidades de respuesta, hospitales, tráfico, semáforos e incidentes viales, y de generar rutas prioritarias y alternativas que minimicen el tiempo estimado de llegada o de traslado, con seguimiento y recálculo en tiempo real mediante simulación.

2.2 Objetivos específicos

1. Implementar la gestión de usuarios, roles y permisos para controlar el acceso a los módulos.
2. Registrar y administrar emergencias con ubicación, tipo, severidad, estado y descripción.
3. Administrar estaciones de emergencia, ambulancias, vehículos de bomberos y su disponibilidad.
4. Registrar hospitales, sus servicios disponibles y su capacidad de atención de urgencias.
5. Construir una representación de la red vial de Valledupar mediante nodos y tramos.
6. Gestionar información de tráfico, semáforos, cierres y otros incidentes que afecten el costo de una ruta.
7. Desarrollar un motor de generación de rutas basado en algoritmos de caminos mínimos, con prioridad para emergencias.
8. Generar una ruta recomendada y dos alternativas, mostrando distancia, tiempo estimado y causas de penalización.
9. Asignar unidades a emergencias y registrar el seguimiento de su ubicación.
10. Implementar auditoría, alertas, notificaciones y reportes operativos.
11. Utilizar Oracle y PL/SQL mediante secuencias, restricciones, índices, vistas, procedimientos y triggers.
12. Mantener una arquitectura modular que permita reemplazar los datos simulados por GPS y fuentes externas de tráfico.

3. Requerimientos

3.1 Requerimientos funcionales

ID	Módulo	Descripción	Casos de uso
RF-01	Autenticación	Permitir inicio de sesión y validación del rol.	CU-01, CU-02, CU-03
RF-02	Emergencias	Crear, consultar, modificar y cerrar emergencias.	CU-06, CU-07, CU-12, CU-15
RF-03	Unidades	Consultar disponibilidad y asignar vehículos.	CU-04, CU-09
RF-04	Hospitales	Consultar hospitales y servicios disponibles.	CU-05, CU-11
RF-05	Rutas	Generar ruta principal y alternativas.	CU-08, CU-17
RF-06	Tráfico	Registrar y consultar niveles de congestión.	CU-08, CU-18, CU-19
RF-07	Semáforos	Registrar estado y aplicar penalización en el cálculo.	CU-08, CU-18
RF-08	Incidentes	Registrar cierres, obras y accidentes viales.	CU-13, CU-14, CU-22
RF-09	Seguimiento	Guardar posiciones y estados de las unidades.	CU-09, CU-10, CU-18, CU-20
RF-10	Alertas	Notificar cambios relevantes al operador.	CU-17
RF-11	Reportes	Generar indicadores y consultas operativas.	CU-16
RF-12	Auditoría	Registrar acciones críticas del sistema.	CU-21 y transversal

3.2 Requerimientos no funcionales

Atributo	Requerimiento	Cómo se cumple
Seguridad	Contraseñas con hash y control de acceso por rol. Credenciales fuera del repositorio.	PBKDF2 con sal (`PasswordHasher`); permisos por rol en `ROL_PERMISO`; `db.properties` excluido de Git
Integridad	Datos consistentes aunque falle una operación.	Transacciones (`TransactionManager`); llaves foráneas, CHECK e índices únicos en Oracle
Rendimiento	El cálculo de una ruta debe ser rápido sobre la red cargada.	Grafo en memoria; A* con heurística admisible; medición en ms contra Dijkstra
Disponibilidad	Manejar fallos de conexión de forma controlada.	Excepciones propias (`DataAccessException`) y mensajes claros sin detalles técnicos
Usabilidad	Ver la ruta recomendada y el estado de la emergencia sin navegar por muchas pantallas.	Pantalla principal con mapa central, indicadores y panel de emergencias
Mantenibilidad	Separación View / Controller / Service / DAO / Model.	Reglas de dependencia verificadas por `ArchitectureTest`
Trazabilidad	Operaciones críticas y cambios de estado auditados.	Tabla `AUDITORIA` y `AuditService`
Veracidad de datos	Distinguir datos simulados de datos reales.	Columnas `fuente` y `origen` con valor `SIMULACION`
Verificabilidad	Cada funcionalidad se comprueba antes de continuar.	Pruebas JUnit y scripts de verificación de la base de datos

4. Análisis: casos de uso

Esta sección define **qué hace el sistema y quién lo usa**. Cada caso de uso indica las clases que lo implementan, para que el código siga el análisis.

4.1 Actores

Actor	Tipo	Descripción	Rol en la base de datos
Operador de Central	Principal	Recibe el reporte, registra la emergencia, consulta rutas y registra incidentes viales	`OPERADOR_CENTRAL`
Coordinador de Emergencias	Principal	Todo lo del operador, más despachar unidades, actualizar la misión, cerrar emergencias y controlar la simulación	`COORDINADOR`
Bombero / Paramédico	Principal	Personal en la unidad: consulta su misión asignada y la ruta	`BOMBERO_PARAMEDICO`
Administrador	Principal	Gestiona usuarios y catálogos (unidades, estaciones, hospitales) y consulta la auditoría	`ADMINISTRADOR`
Simulador	Secundario (sistema, temporizado)	Cada pocos segundos cambia el tráfico simulado y mueve las unidades simuladas. En la fase 2 lo reemplazan una fuente real de tráfico y el GPS	—
Ciudadano	Secundario (fuera del núcleo)	Reporta un incidente vial escaneando un código QR	—

El **Coordinador** hereda los casos de uso del **Operador** (generalización de actores).

4.2 Diagramas de casos de uso

Se presenta un diagrama por actor para que se lean sin cruces. Los casos de uso aparecen en óvalos dentro del sistema y las relaciones `include` y `extend` con líneas punteadas.

4.2.1 Operador de Central

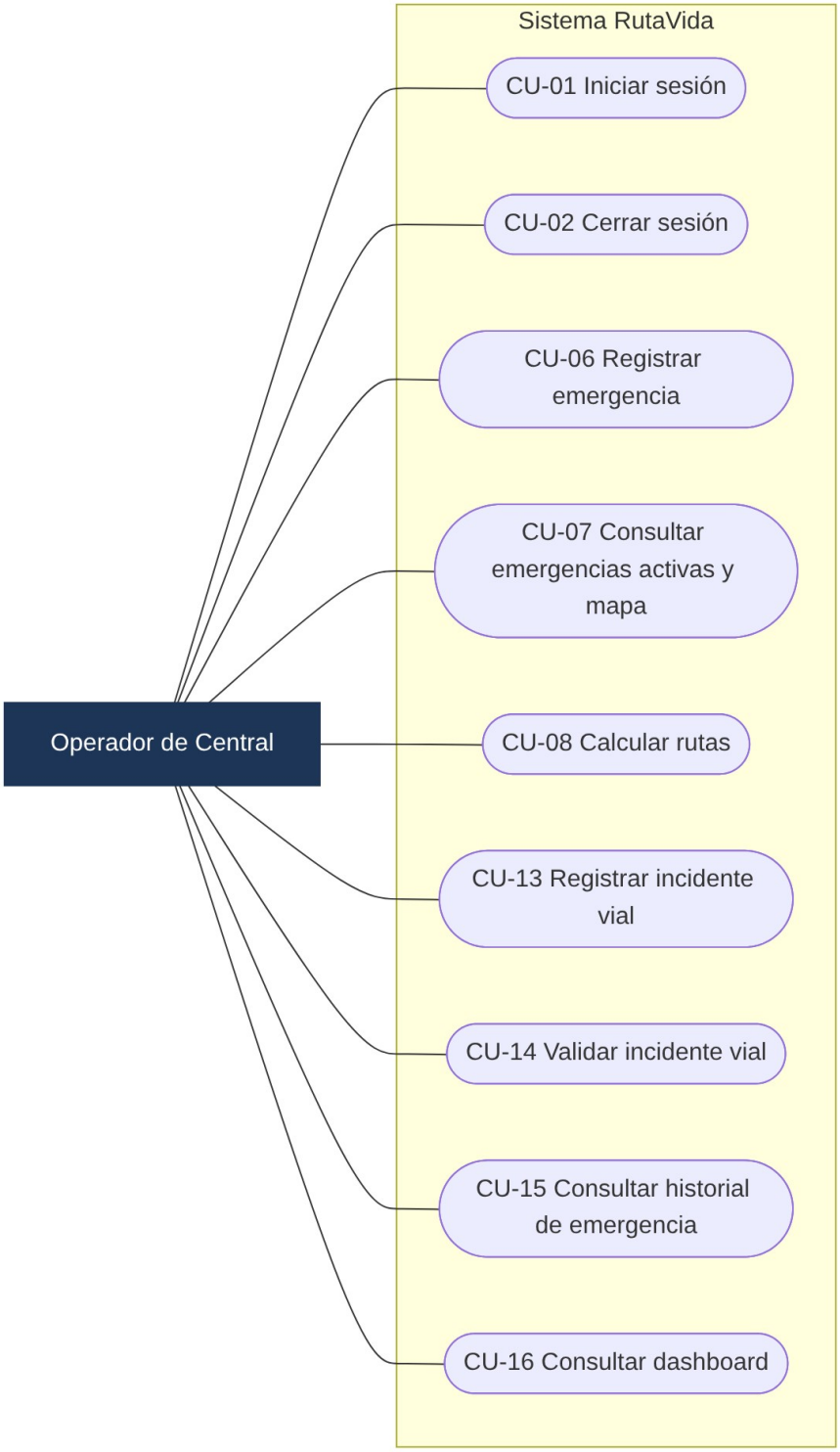


Figura 1. Casos de uso — Operador de Central

4.2.2 Coordinador de Emergencias

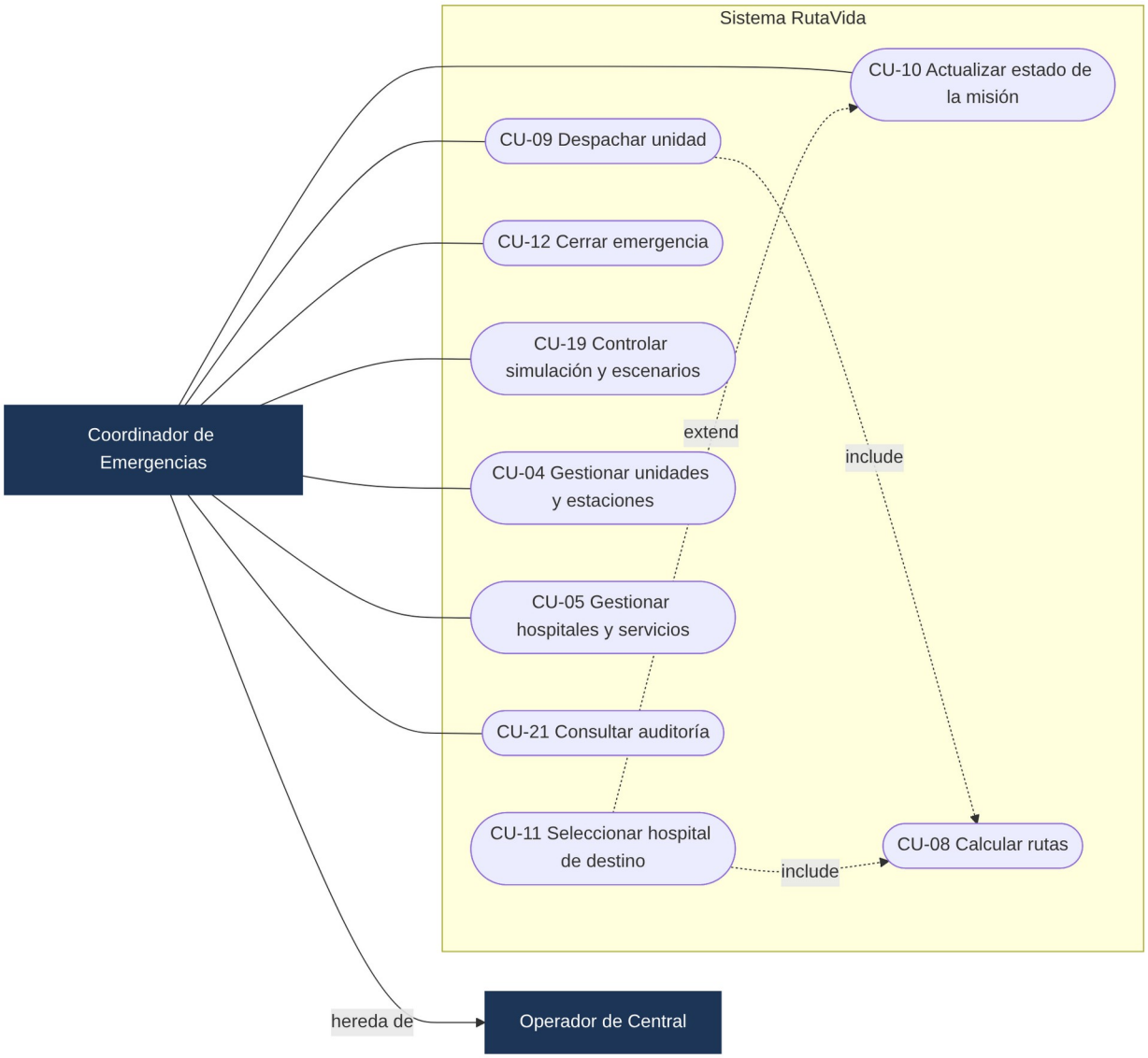


Figura 2. Casos de uso — Coordinador de Emergencias

4.2.3 Bombero / Paramédico, Administrador y Ciudadano

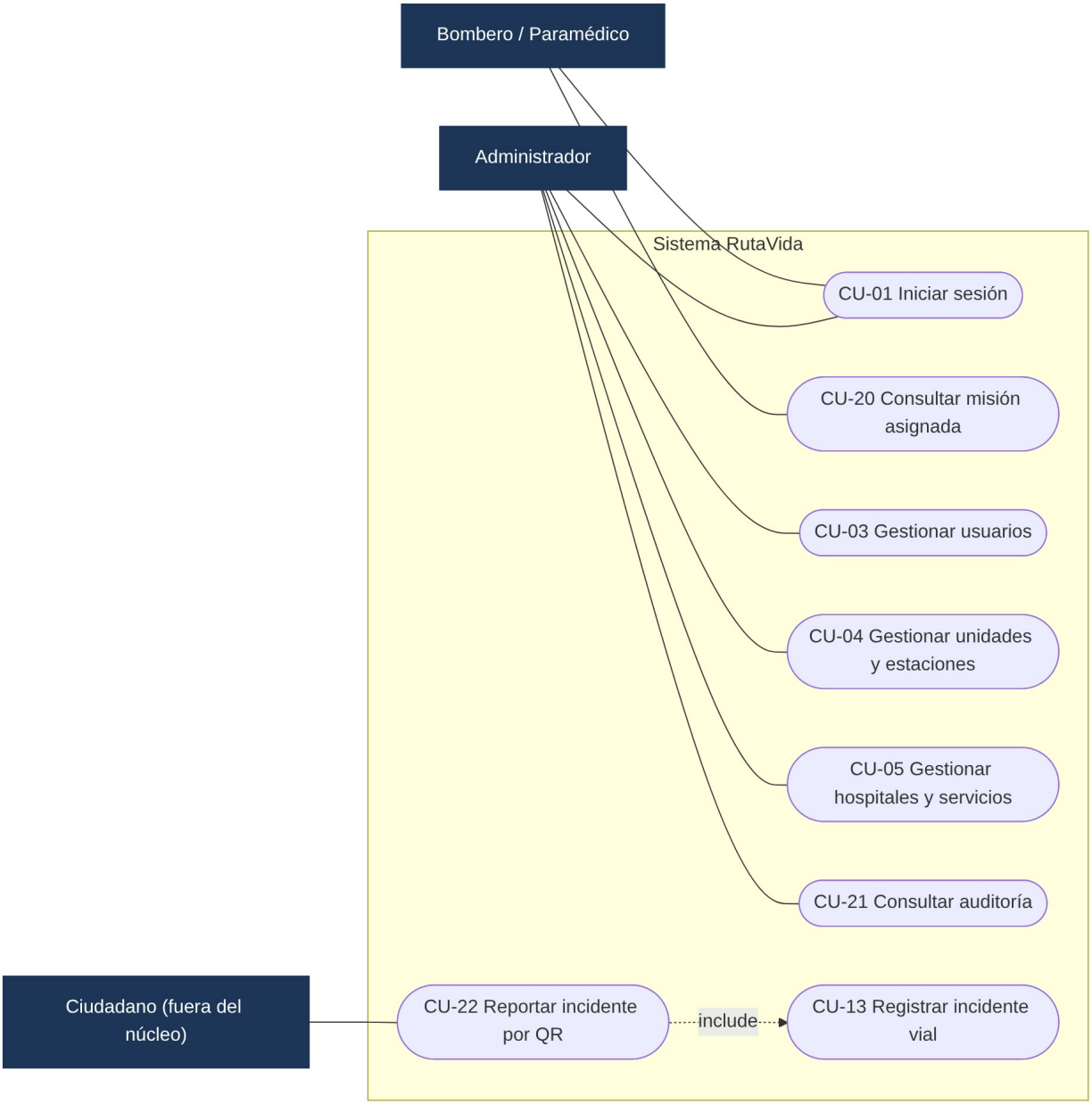


Figura 3. Casos de uso — Bombero / Paramédico, Administrador y Ciudadano

4.2.4 Simulador y recálculo automático

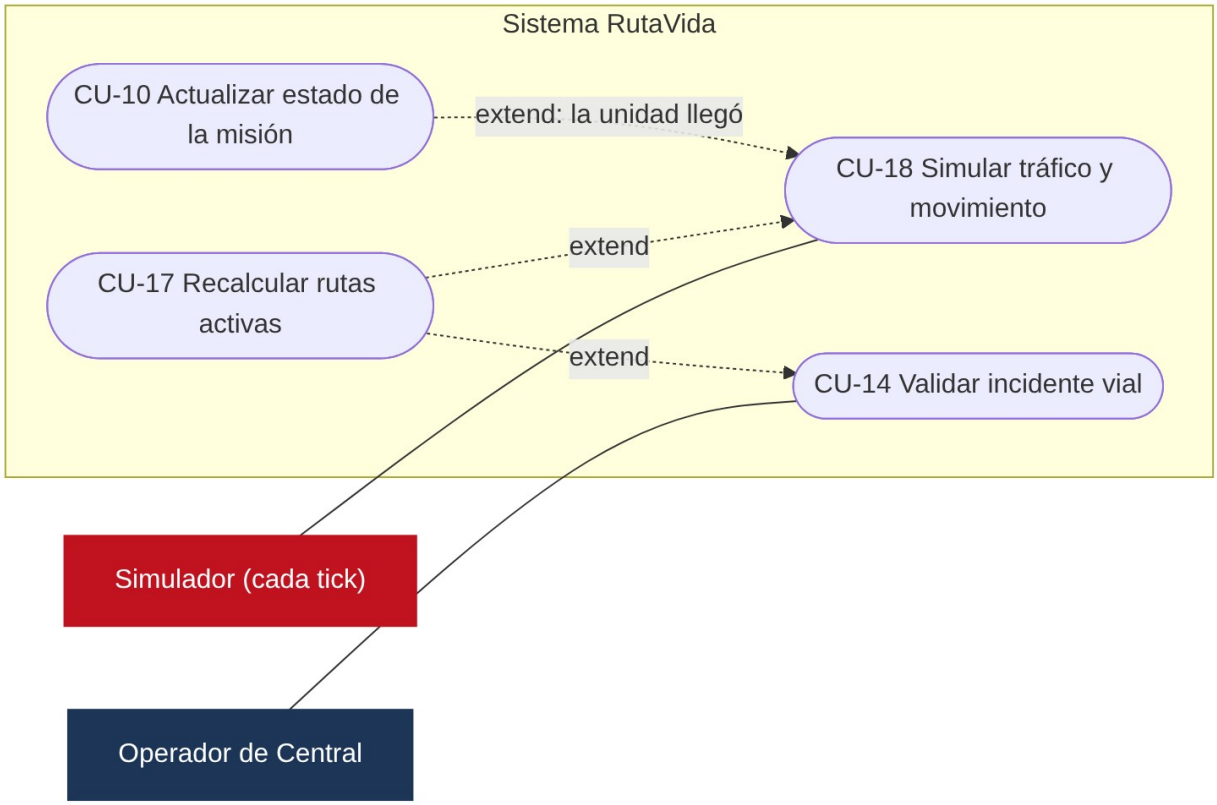


Figura 4. Casos de uso — Simulador y recálculo automático

4.3 Lista de casos de uso

Prioridad: N = núcleo de la primera versión (lo que se demuestra) · C = complemento · F = fase posterior.

ID	Caso de uso	Actor(es)	Permiso requerido	RF	Prioridad
CU-01	Iniciar sesión	Todos	—	RF-01	N
CU-02	Cerrar sesión	Todos	—	RF-01	N
CU-03	Gestionar usuarios	Administrador	`USUARIOS/GESTIONAR`	RF-01	C
CU-04	Gestionar unidades y estaciones	Administrador, Coordinador	`UNIDADES/GESTIONAR`	RF-03	C
CU-05	Gestionar hospitales y servicios	Administrador, Coordinador	`HOSPITALES/GESTIONAR`	RF-04	C
CU-06	Registrar emergencia	Operador	`EMERGENCIAS/GESTIONAR`	RF-02	N
CU-07	Consultar emergencias activas y mapa	Operador, Bombero	`EMERGENCIAS/CONSULTAR`	RF-02	N
CU-08	Calcular rutas (principal + 2 alternativas)	Operador	`RUTAS/GESTIONAR`	RF-05, RF-06, RF-07	N

ID	Caso de uso	Actor(es)	Permiso requerido	RF	Prioridad
CU-09	Despachar unidad	Coordinador	`DESPACHO/GESTIONAR`	RF-03, RF-09	N
CU-10	Actualizar estado de la misión	Coordinador	`DESPACHO/GESTIONAR`	RF-09	N
CU-11	Seleccionar hospital de destino	Coordinador	`DESPACHO/GESTIONAR`	RF-04	N
CU-12	Cerrar emergencia	Coordinador	`DESPACHO/GESTIONAR`	RF-02	N
CU-13	Registrar incidente vial	Operador	`INCIDENTES/GESTIONAR`	RF-08	N
CU-14	Validar incidente vial (activar o rechazar)	Operador	`INCIDENTES/GESTIONAR`	RF-08	N
CU-15	Consultar historial de emergencia	Operador	`EMERGENCIAS/CONSULTAR`	RF-02, RF-12	N
CU-16	Consultar dashboard	Operador	`REPORTES/CONSULTAR`	RF-11	C
CU-17	Recalcular rutas activas	Sistema	— (automático)	RF-05, RF-10	N
CU-18	Simular tráfico y movimiento	Simulador	— (automático)	RF-06, RF-07, RF-09	N
CU-19	Controlar simulación y escenarios de demostración	Coordinador	`TRAFICO/GESTIONAR`	RF-06	N
CU-20	Consultar misión asignada	Bombero / Paramédico	`DESPACHO/CONSULTAR`	RF-09	C
CU-21	Consultar auditoría	Administrador, Coordinador	`AUDITORIA/CONSULTAR`	RF-12	C
CU-22	Reportar incidente por QR	Ciudadano	Token firmado del QR	RF-08	F

Ajuste de permisos: CU-12 (cerrar emergencia) exige `DESPACHO/GESTIONAR`, que solo tienen el Coordinador y el Administrador. Así el Operador registra, pero no cierra, emergencias. Coincide con los datos de `02_datos_iniciales.sql`.

Transversal a todos los casos (RF-12): toda operación que cambia datos y todo inicio de sesión (exitoso o fallido) se registra en `AUDITORIA`.

4.4 Especificación de los casos del núcleo

CU-01 · Iniciar sesión

Campo	Contenido
Actor	Cualquier usuario registrado
Precondición	El usuario existe en `USUARIO`

Campo	Contenido
Postcondición	Hay una sesión activa con el usuario, su rol y sus permisos; queda un registro en `AUDITORIA`

Flujo principal

1. El usuario escribe su nombre de usuario y su contraseña.
2. El sistema busca el usuario por `username`.
3. El sistema compara la contraseña con `password_hash` (PBKDF2, nunca texto plano).
4. El sistema verifica que el usuario esté activo (`estado = 'A'`).
5. El sistema carga el rol y sus permisos.
6. El sistema registra `LOGIN / EXITO` en la auditoría.
7. El sistema muestra la pantalla principal con el nombre y el rol del usuario (arriba a la derecha).

Flujos alternos

- 2a / 3a. Usuario inexistente o contraseña incorrecta → mensaje único "Usuario o contraseña incorrectos" (no revela cuál de los dos falló) y auditoría `LOGIN / FALLO`.
- 4a. Usuario inactivo → "El usuario está inactivo" y auditoría `LOGIN / FALLO`.
- *a. Base de datos no disponible → "No hay conexión con la base de datos" sin mostrar detalles técnicos.

Clases: `LoginController` → `AuthService` → `UserDAO`, `RoleDAO`, `PasswordHasher`, `AuditService` → `UserSession`.

CU-06 · Registrar emergencia

Campo	Contenido
Actor	Operador de Central
Precondición	Sesión activa con `EMERGENCIAS/GESTIONAR`
Postcondición	Emergencia en estado `REPORTADA`, auditada y visible en el mapa de todos los operadores

Flujo principal

1. El operador elige el tipo de emergencia (`ACCIDENTE_VIAL`, `INCENDIO`, `EMERGENCIA_MEDICA`, `RESCATE`).
2. Marca la ubicación en el mapa (o escribe las coordenadas) y una dirección de referencia.
3. Elige la severidad (`BAJA`, `MEDIA`, `ALTA`, `CRITICA`) y escribe una descripción opcional.
4. El sistema valida los datos (RN-03).
5. En una sola transacción: inserta la emergencia con estado `REPORTADA` y registra la auditoría.
6. El sistema publica el evento `EmergencyRegistered`; el mapa y la lista de emergencias activas se actualizan.
7. El sistema ofrece continuar con CU-09 (despachar unidad).

Flujos alternos

- 4a. Datos incompletos o coordenadas fuera del área cubierta por la red vial cargada → mensaje con el campo que falla; no se guarda nada.
- 5a. Falla la base de datos → se deshace la transacción y se informa el error.

Clases: `EmergencyController` → `EmergencyService` → `EmergencyDAO`, `EmergencyTypeDAO`, `TransactionManager`, `AuditService`, `EventPublisher`.

CU-08 · Calcular rutas

Campo	Contenido
Actor	Operador (también se incluye desde CU-09 y CU-11)
Precondición	Grafo vial cargado en memoria; origen y destino dentro de la red
Postcondición	Hasta 3 opciones de ruta ordenadas por tiempo, cada una con su explicación. Solo se guardan en `RUTA` cuando se despacha (CU-09)

Flujo principal

1. El sistema recibe el origen (posición de la unidad o del lugar de la emergencia), el destino y el tipo de vehículo.
2. Ubica el nodo del grafo más cercano al origen y al destino.
3. Calcula la ruta más rápida con A*, usando el costo en segundos de cada tramo: tiempo con congestión + espera por semáforos + penalización por incidentes, ajustados al tipo de vehículo (RN-07).
4. Calcula dos alternativas penalizando los tramos ya usados y descarta las que son casi iguales o demasiado lentas (RN-08).
5. Para cada opción arma la explicación: distancia, tiempo, número de semáforos, tramos congestionados e incidentes evitados.
6. Muestra las 3 rutas en el mapa con colores distintos y la recomendada resaltada.

Flujos alternos

- 3a. No existe camino (por ejemplo, por cierres) → "No hay ruta disponible con las condiciones actuales".
- 4a. Solo existen 1 o 2 rutas suficientemente distintas → se muestran las que hay y se indica por qué no hay más.

Clases: `RouteController` → `RouteService` → `RoadGraph`, `AStarRouter`, `AlternativeRouteFinder`, `CostFunction` / `EmergencyCost`, `RouteExplanationBuilder` → `RouteOption` (DTO).

CU-09 · Despachar unidad

Campo	Contenido
Actor	Coordinador de Emergencias
Precondición	Emergencia en estado `REPORTADA`; al menos una unidad `DISPONIBLE`
Postcondición	Asignación `ASIGNADA`, emergencia `ASIGNADA`, unidad `OCUPADO`, ruta principal `ACTIVA` y alternativas `PROPUESTA`; todo auditado

Flujo principal

1. El coordinador abre una emergencia `REPORTADA`.
2. El sistema calcula el tiempo estimado de llegada (ETA) de cada unidad disponible de un tipo adecuado (incluye CU-08) y las ordena de menor a mayor.

3. El coordinador elige una unidad (por defecto, la de menor ETA).
4. El sistema muestra las 3 rutas de esa unidad hasta la emergencia.
5. El coordinador confirma el despacho.
6. En **una sola transacción**: crea la asignación, cambia la emergencia a `ASIGNADA`, cambia la unidad a `OCUPADO`, guarda las rutas en `RUTA` / `RUTA_TRAMO` y registra la auditoría.
7. El sistema publica `UnitDispatched`; la unidad empieza a moverse en el mapa (simulación, CU-18).

Flujos alternos

- 2a. No hay unidades disponibles → "No hay unidades disponibles" y la emergencia sigue `REPORTADA`.
- 6a. Otro coordinador asignó la misma unidad un instante antes → la base de datos lo impide (índice único de asignación abierta) y el sistema pide elegir otra unidad.
- 6b. Falla cualquier escritura → se deshace todo.

Varias emergencias a la vez (extensión): con N emergencias pendientes y M unidades, `DispatchService` arma la matriz de ETA y la resuelve con el algoritmo húngaro (asignación óptima, Investigación de Operaciones).

Clases: `DispatchController` → `DispatchService` → `EmergencyDAO`, `EmergencyVehicleDAO`, `EmergencyAssignmentDAO`, `RouteService`, `RouteDAO`, `HungarianAlgorithm`, `TransactionManager`, `AuditService`, `EventPublisher`.

CU-10 · Actualizar estado de la misión (incluye CU-11)

Campo	Contenido
Actor	Coordinador (la llegada al sitio también la puede marcar automáticamente el simulador)
Precondición	Asignación abierta
Postcondición	Asignación y emergencia con el mismo estado nuevo; horas registradas; auditoría

Flujo principal

1. El coordinador elige la siguiente fase: `EN_RUTA` → `EN_SITIO` → `TRASLADANDO` → `FINALIZADA`.
2. El sistema valida la transición (RN-01, RN-02).
3. En una transacción: actualiza la asignación (`hora_salida` al pasar a `EN_RUTA`, `hora_llegada` al pasar a `EN_SITIO`), la emergencia y la auditoría.
4. Publica `EmergencyStatusChanged`.

Extensión CU-11 · Seleccionar hospital (al pasar a `TRASLADANDO`)

1. El sistema lista los hospitales operativos que tienen el servicio requerido.
2. Los ordena por costo = ETA del traslado + penalización por baja disponibilidad (RN-10).
3. El coordinador confirma uno; el sistema calcula las rutas de traslado (CU-08) y las guarda con `id_hospital_destino`.

Flujos alternos

- 2a. Transición no permitida (por ejemplo, de `ASIGNADA` a `TRASLADANDO`) → mensaje y no se cambia nada.
- 1a. La unidad se avería → la asignación pasa a `CANCELADA`, la unidad a `MANTENIMIENTO`, la emergencia vuelve a `REPORTADA` y se repite CU-09.

Clases: `DispatchController` → `DispatchService`, `HospitalService`, `RouteService`.

CU-12 · Cerrar emergencia

Campo	Contenido
Actor	Coordinador
Precondición	Emergencia `EN_SITIO` o `TRASLADANDO`, o `REPORTADA` si fue una falsa alarma
Postcondición	Emergencia `CERRADA` con `fecha_hora_cierre`; asignación `FINALIZADA`; unidad `DISPONIBLE`; rutas `FINALIZADA`

Flujo principal

1. El coordinador elige "Cerrar emergencia".
2. El sistema valida la transición.
3. En una transacción: cierra la emergencia, finaliza la asignación y las rutas activas, libera la unidad y audita.
4. El sistema calcula los tiempos de respuesta (asignación → salida → llegada → cierre) para el dashboard.
5. Publica `EmergencyStatusChanged`.

Clases: `DispatchController` → `DispatchService` → `EmergencyDAO`, `EmergencyAssignmentDAO`, `EmergencyVehicleDAO`, `RouteDAO`, `TransactionManager`, `AuditService`, `EventPublisher`.

CU-14 · Validar incidente vial

Campo	Contenido
Actor	Operador
Precondición	Incidente en estado `PENDIENTE` (creado en CU-13, por QR o por un escenario de demostración)
Postcondición	Incidente `ACTIVO` (afecta las rutas) o `RECHAZADO` (no las afecta)

Flujo principal

1. El operador revisa el incidente pendiente en el mapa.
2. Lo activa e indica si bloquea el tramo (`bloquea_tramo = S`) o solo lo penaliza.
3. El sistema actualiza el estado del tramo en el grafo en memoria (RN-05).
4. Publica `IncidentActivated` → se dispara CU-17 para las rutas activas que pasan por ese tramo.

Flujo alterno

- 2a. El operador lo rechaza → estado `RECHAZADO`; el grafo no cambia.

Clases: `IncidentController` → `IncidentService` → `RoadIncidentDAO`, `RoadConditionService`, `EventPublisher`.

CU-17 · Recalcular rutas activas (automático)

Campo	Contenido
Actor	Sistema (lo disparan CU-14 y CU-18)
Precondición	Hay al menos una ruta `ACTIVA`
Postcondición	Si hubo una mejora suficiente, la nueva ruta queda `ACTIVA`, la anterior `DESCARTADA`, y el coordinador recibe una alerta

Flujo principal

1. Llega un evento de cambio en la vía (`TrafficChanged` o `IncidentActivated`).
2. El sistema busca las rutas activas que pasan por los tramos afectados.
3. Para cada una, calcula la mejor ruta desde la **posición actual** de la unidad hasta su destino.
4. Si la nueva ruta mejora el tiempo restante por encima del umbral (RN-09) o la ruta actual quedó bloqueada:
    - en una transacción, guarda la nueva ruta como `ACTIVA` y marca la anterior `DESCARTADA`;
    - crea una alerta `RECALCULO_RUTA` para el coordinador;
    - publica `RouteRecalculated`; el mapa dibuja la nueva ruta y muestra el tiempo ganado.

Flujo alterno

- 4a. La mejora no supera el umbral → no se cambia nada (evita que la ruta "parpadee").

Clases: `RouteMonitor` → `RouteService`, `RouteDAO`, `AlertService`, `EventBus`.

CU-18 · Simular tráfico y movimiento (automático)

Campo	Contenido
Actor	Simulador (cada *tick*, por defecto cada 5 s; configurable)
Precondición	Simulación iniciada (CU-19)
Postcondición	Grafo con congestión y semáforos actualizados; unidades movidas; historial guardado por lotes

Flujo principal (cada tick)

1. El proveedor de tráfico simulado genera cambios de congestión y de semáforos (nunca cierra tramos: RN-06).
2. `RoadConditionService` combina tráfico + incidentes y actualiza el grafo.
3. El simulador de movimiento avanza cada unidad en misión a lo largo de su ruta activa.
4. Se publican `TrafficChanged` y `VehiclePositionUpdated`; el mapa se actualiza.
5. Cada N ticks se guardan `TRAFICO` y `UBICACION_VEHICULO` con fuente `SIMULACION` (RN-11).
6. Si una unidad llega a su destino, el sistema pasa la misión a `EN_SITIO` (CU-10).

Extensión: si los cambios afectan una ruta activa → CU-17.

Clases: `SimulationEngine` → `RoadConditionService` (con `TrafficProvider` / `SimulatedTrafficProvider`), `TrackingService` (con `VehiclePositionSource` / `VehicleMovementSimulator`).

4.5 Reglas de negocio

ID	Regla	Dónde se garantiza
RN-01	Una emergencia solo cambia de estado siguiendo el flujo `REPORTADA → ASIGNADA → EN_RUTA → EN_SITIO → (TRASLADANDO) → CERRADA` (más el regreso a `REPORTADA` si se cancela la asignación y el cierre directo por falsa alarma)	`EmergencyStatus.canChangeTo()` (programado)

ID	Regla	Dónde se garantiza
RN-02	Solo se despachan unidades `DISPONIBLE`, y una unidad tiene como máximo una asignación abierta	`VehicleOperationalStatus.isDispatchable()` + índice único `ux_asignacion_vehiculo_abierta`
RN-03	Una emergencia exige tipo, coordenadas válidas dentro de la red vial cargada, dirección de referencia y severidad	`EmergencyService` + CHECK de Oracle
RN-04	Las contraseñas se guardan con hash PBKDF2 y sal; nunca en texto plano	`PasswordHasher`
RN-05	Un incidente nuevo queda `PENDIENTE` y solo los `ACTIVO` afectan las rutas	`IncidentStatus.affectsRouting()` + DEFAULT en Oracle
RN-06	El simulador de tráfico cambia la congestión y los semáforos, pero no cierra tramos; los cierres vienen solo de incidentes	`TrafficProvider` / `RoadConditionService`
RN-07	El costo de un tramo se mide en segundos e incluye congestión, semáforos e incidentes, ajustado al tipo de vehículo	`EmergencyCost`
RN-08	Las alternativas se descartan si comparten más del 80 % de los tramos con otra ruta o tardan más de 1,5 veces la mejor	`AlternativeRouteFinder`
RN-09	Una ruta activa se reemplaza solo si la nueva mejora el tiempo restante en más del 15 % o en más de 30 s, o si la actual quedó bloqueada	`RouteMonitor`
RN-10	El hospital de traslado debe estar operativo y tener el servicio requerido; se elige por menor ETA con penalización por baja disponibilidad	`HospitalService`
RN-11	Los datos simulados se marcan `SIMULACION` y nunca se presentan como reales; el historial se guarda por lotes	Enums `TrafficSource`, `PositionSource`
RN-12	Toda operación que modifica datos y todo inicio de sesión se audita	`AuditService`

Los valores de RN-08 y RN-09 (80 %, 1,5×, 15 %, 30 s) son **parámetros iniciales a calibrar** con pruebas, no datos medidos.

4.6 Escenario de la demostración

1. El operador inicia sesión (CU-01) y registra un accidente en Valledupar (CU-06).
2. El coordinador despacha la ambulancia con menor ETA; se ven 3 rutas con su explicación (CU-09, CU-08).
3. La ambulancia empieza a moverse en el mapa (CU-18).
4. El coordinador ejecuta el escenario "accidente en la vía de la ruta recomendada" (CU-19 → CU-13 → CU-14).
5. El sistema recalcula y muestra la nueva ruta con el tiempo ganado (CU-17).
6. La unidad llega, traslada al paciente al hospital elegido y se cierra la emergencia (CU-10, CU-11, CU-12).
7. Se consulta el historial completo y el dashboard (CU-15, CU-16).

5. Arquitectura de software

5.1 Tecnologías

Componente	Tecnología	Observación
Lenguaje	Java (compilado para Java 21)	En el equipo de desarrollo se usa el JDK 25, compatible
Construcción	Maven	`backend/pom.xml`
Base de datos	Oracle Database XE 18c	Local en cada equipo: `localhost:1521/XEPDB1`, usuario `RUTAVIDA`
Acceso a datos	JDBC con `ojdbc11` 21.9.0.0	Compatible con Oracle 18c, 19c y 21c
Pruebas	JUnit 5.11.3	37 pruebas automáticas
IDE y versiones	IntelliJ IDEA, Git y GitHub	Rama `develop` con Pull Requests
Diagramas	Mermaid	En archivos `.md` del repositorio
Vista	**Pendiente**	Java Swing o web (JavaScript + Leaflet), según el profesor

5.2 Idea general

RutaVida usa una **arquitectura en capas** con tres decisiones que la hacen defendible:

1. **Las capas superiores dependen de interfaces, no de implementaciones.** Un servicio conoce `EmergencyDAO`, no `JdbcEmergencyDAO`. La única clase que conecta interfaces con implementaciones es `app.AppContext` (*composition root*, inyección de dependencias manual, sin framework).
2. **El tiempo real se resuelve con eventos.** El simulador y los servicios publican eventos (`TrafficChanged`, `RouteRecalculated`…) y quien los necesite se suscribe. Nadie llama directamente a la vista.
3. **Las reglas se verifican con pruebas.** `ArchitectureTest` falla si alguien rompe una regla de dependencia, y `EnumSqlContractTest` falla si los estados de Java y los CHECK de Oracle dejan de coincidir.

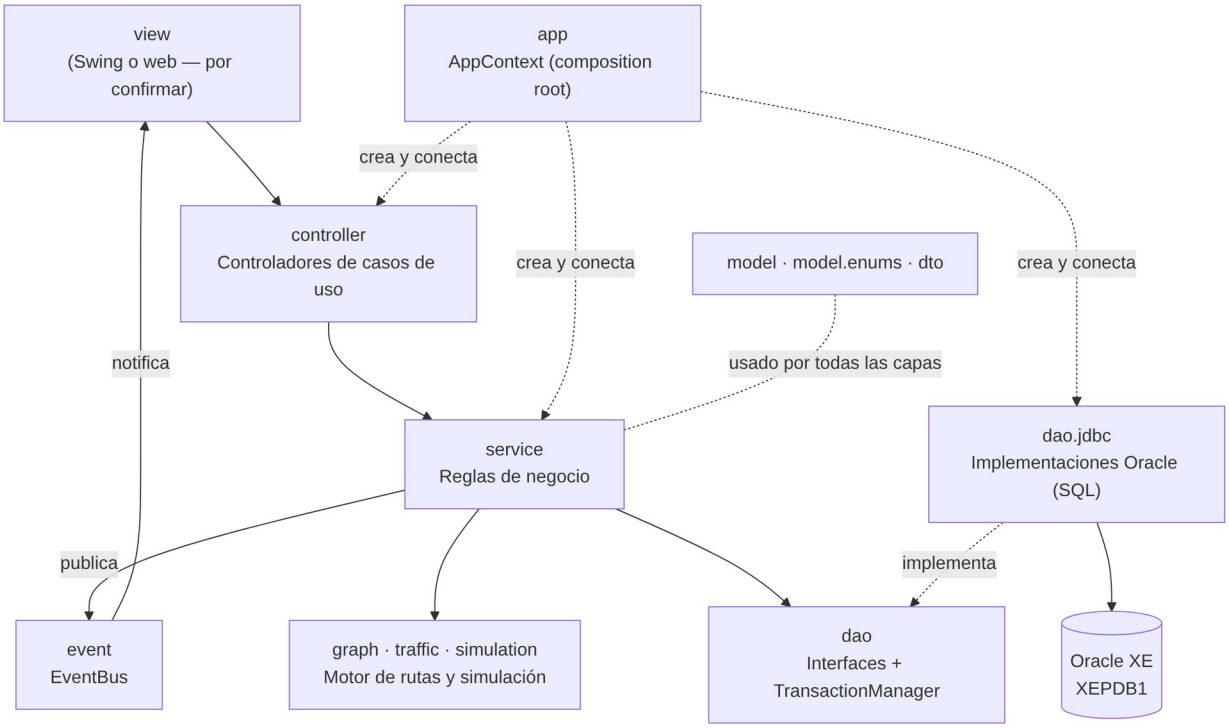


Figura 5. Arquitectura en capas de RutaVida

5.3 Paquetes

Paquete raíz: `co.edu.unicesar.rutavida`. Cada paquete tiene un archivo `package-info.java` con su responsabilidad, sus dependencias permitidas y su dueño.

Paquete	Capa	Responsabilidad	Dueño
`app`	Arranque	`AppContext`: crea los objetos y les inyecta sus dependencias. Puntos de entrada	Andrés
`model`	Model	Entidades, una por tabla, con nombres en inglés (`Emergency`, `RoadSegment`…)	Sherly
`model.enums`	Model	Estados y catálogos. Cada constante en inglés guarda su código en español de Oracle	Sherly (códigos: Andrés)
`dto`	Model	Objetos de transferencia (records) entre el controlador y la vista: opción de ruta con explicación, resumen del dashboard	Sherly
`dao`	DAO	**Interfaces** de persistencia (una por agregado) y `TransactionManager`. Sin tipos JDBC	Andrés
`dao.jdbc`	DAO	Implementaciones con JDBC y SQL de Oracle, transacciones, mapeo fila → objeto	Andrés
`service`	Service	Reglas de negocio: validar, aplicar reglas, coordinar DAOs en una transacción y publicar eventos	Kendrick (rutas, despacho, seguimiento) · Sherly (usuarios, hospitales, alertas, auditoría)

Paquete	Capa	Responsabilidad	Dueño
`graph`	Service	Motor de rutas en memoria: grafo, función de costo, A*/Dijkstra, rutas alternativas	Kendrick
`traffic`	Service	Fuentes de tráfico detrás de `TrafficProvider`: simulador (fase 1) y fuente externa (fase 2)	Kendrick
`simulation`	Service	Ciclo de tiempo real, movimiento simulado de unidades y escenarios de demostración	Kendrick
`event`	Transversal	Publicación y suscripción de eventos para el tiempo real	Andrés
`controller`	Controller	Controladores de casos de uso: reciben la acción del usuario y delegan en los servicios	Sherly
`view`	View	Interfaz de usuario. **Tecnología pendiente (Swing o web)**	Diego
`util`	Util	Configuración de la conexión, hash de contraseñas, validaciones y mensajes en español	Sherly (`DatabaseConnection`: Andrés)
`exception`	Transversal	Jerarquía de excepciones compartida	Andrés

5.4 Reglas de dependencia

Las reglas se verifican automáticamente con la prueba `ArchitectureTest`: si una capa importa algo que no le corresponde, la prueba falla e indica el archivo.

Paquete	No puede importar
`model`, `model.enums`	`java.sql`, Swing/AWT, `dao`, `service`, `controller`, `view`, `graph`, `traffic`, `simulation`, `event`, `app`
`dao` (interfaces)	`java.sql`, `javax.sql`, `oracle.*`, `dao.jdbc`, `service`, `controller`, `view`
`dao.jdbc`	Swing/AWT, `service`, `controller`, `view`, `simulation`, `app`
`service`	`java.sql`, `javax.sql`, `oracle.*`, Swing/AWT, `dao.jdbc`, `controller`, `view`, `app`
`graph`, `traffic`	JDBC, Swing/AWT, `dao`, `service`, `controller`, `view`, `simulation`, `app`
`event`	JDBC, Swing/AWT, `dao`, `service`, `controller`, `view`, `app`
`controller`	JDBC, Swing/AWT, `dao`, `view`, `app`
`view`	JDBC, `dao`, `service`, `graph`, `traffic`, `simulation`
Todos, salvo `dao.jdbc` y `app`	`dao.jdbc`

En resumen: **la vista solo habla con controladores, los controladores solo con servicios, y solo los DAO hablan con Oracle.**

5.5 Flujo de un caso de uso: despachar una unidad

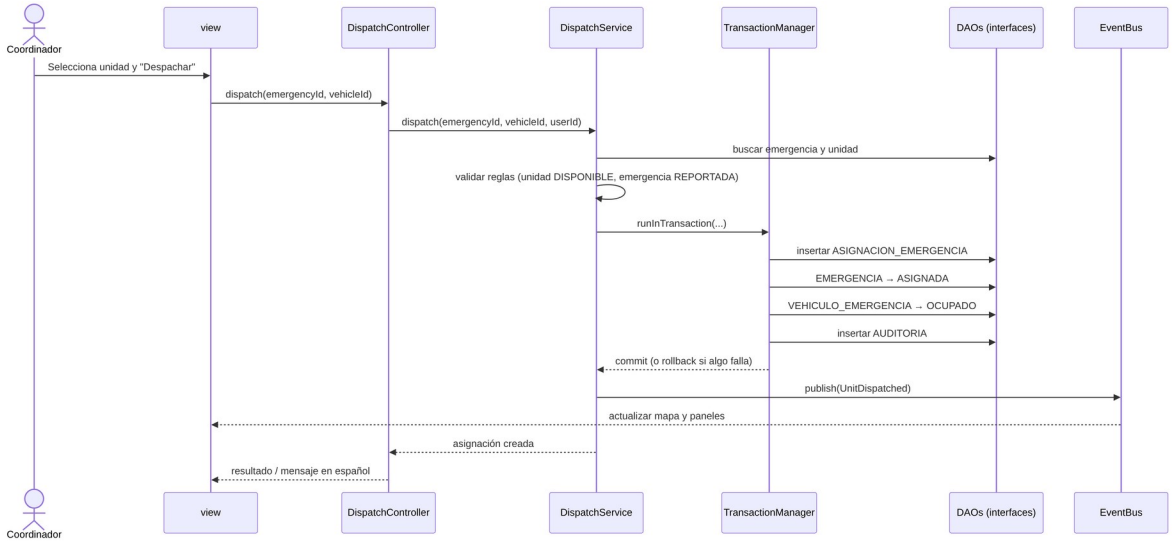


Figura 6. Secuencia del despacho de una unidad, con transacción y evento

- **Transacción:** las cuatro escrituras se guardan juntas o no se guarda ninguna. Los DAOs no hacen `commit` ni `close`; de eso se encarga `JdbcTransactionManager`.
- **Errores:** si una regla no se cumple, el servicio lanza `BusinessRuleException`; si falla Oracle, el DAO lanza `DataAccessException`. El controlador convierte ambas en un mensaje en español para el usuario.

5.6 Flujo del tiempo real

1. En cada *tick* del simulador, `TrafficProvider` entrega los cambios de tráfico y el grafo en memoria se actualiza.
2. Se publica el evento `TrafficChanged`.
3. `RouteMonitor` recalcula las rutas **activas** que pasan por los tramos afectados.
4. Si la mejora supera el umbral, se publica `RouteRecalculated` y la vista redibuja la ruta.

- El historial (`TRAFICO`, `UBICACION_VEHICULO`) se guarda **por lotes**, no en cada tick.
- `InMemoryEventBus` entrega los eventos en el hilo que publica. Una vista Swing debe actualizar la pantalla con `SwingUtilities.invokeLater`; una vista web reenviaría el evento al navegador.

5.7 GRASP aplicado a RutaVida

Patrón	Dónde se aplica	Estado
Information Expert	`EmergencyStatus.canChangeTo()`, `AssignmentStatus.isOpen()`, `IncidentStatus.affectsRouting()`, `VehicleOperationalStatus.isDispatchable()`: cada estado sabe sus propias reglas	Programado
Information Expert	`RoadGraph` conoce sus tramos y su estado; la entidad `Route` calcula su tiempo total	Por programar
Creator	`DispatchService` crea la `EmergencyAssignment` (tiene la emergencia, la unidad y el usuario); `RouteService` crea las `Route`	Por programar

Patrón	Dónde se aplica	Estado
Controller	`LoginController`, `EmergencyController`, `DispatchController`… reciben los eventos del sistema que genera la vista	Por programar
Low Coupling	La vista solo conoce controladores; los servicios solo interfaces DAO; los eventos desacoplan a quien publica de quien escucha	Reglas programadas y probadas
High Cohesion	Un servicio por área de negocio (`EmergencyService`, `DispatchService`, `RouteService`…); un DAO por agregado	Estructura definida
Polymorphism	`CostFunction` (costo de emergencia, solo distancia…), `TrafficProvider` (simulado o real)	Diseñado en el documento consolidado
Pure Fabrication	`JdbcTransactionManager`, `AppContext`, `DatabaseConnection`, los DAOs, `InMemoryEventBus`, `GraphLoader`	Los cuatro primeros programados
Indirection	`TransactionManager` entre servicios y JDBC; `EventBus` entre simulador, servicios y vista; los controladores entre vista y servicios	Programado
Protected Variations	Interfaces DAO (cambiar de Oracle local a la nube o a otra base), `TrafficProvider` (simulación → fuente real), `ConnectionFactory`, `Clock`	Programado en DAO y conexión

5.8 SOLID aplicado a RutaVida

Principio	Cómo se cumple	Ejemplo
S — Responsabilidad única	Cada clase tiene un solo motivo para cambiar: el SQL solo cambia en `dao.jdbc`, las reglas en `service`, los textos en `util`	`JdbcTransactionManager` solo maneja transacciones; `DatabaseConnection` solo la configuración
O — Abierto/cerrado	Se agregan comportamientos con clases nuevas sin modificar las existentes	Nueva `CostFunction` o nuevo `TrafficProvider` sin tocar `AStarRouter`
L — Sustitución de Liskov	Toda implementación respeta el contrato de su interfaz	Un `TrafficProvider` real o simulado nunca devuelve `null` y solo reporta tramos que existen; un DAO falso de prueba se comporta como el real
I — Segregación de interfaces	Interfaces pequeñas: no hay un `CrudDAO` genérico con métodos que no aplican	`TRAFICO`, `UBICACION_VEHICULO` y `AUDITORIA` son solo de inserción: sus DAOs no tienen `update` ni `delete`. Los servicios que solo publican usan `EventPublisher`, no todo el `EventBus`
D — Inversión de dependencias	Los módulos de alto nivel dependen de abstracciones y `AppContext` elige la implementación	`DispatchService(EmergencyDAO, VehicleDAO, TransactionManager, EventPublisher, Clock)`

5.9 Convenciones de nombres y de código

Elemento	Convención	Ejemplo
Entidad	Sustantivo en inglés, singular (tabla V.4 del documento consolidado)	`Emergency`, `RoadSegment`, `EmergencyAssignment`

Elemento	Convención	Ejemplo
Enum de estado	`<Entidad>Status` o nombre del catálogo, implementa `CodedEnum`	`EmergencyStatus.EN_ROUTE` → `"EN_RUTA"`
Interfaz DAO	`<Entidad>DAO`	`EmergencyDAO`
Implementación DAO	`Jdbc<Entidad>DAO` en `dao.jdbc`	`JdbcEmergencyDAO`
Servicio	`<Área>Service`	`DispatchService`, `RouteService`
Controlador	`<Área>Controller`	`EmergencyController`
DTO	Record con nombre descriptivo	`RouteOptionDto`, `DashboardSummaryDto`
Evento	Record en pasado	`EmergencyStatusChanged`, `RouteRecalculated`
Excepción	`<Motivo>Exception`, hereda de `RutaVidaException`	`BusinessRuleException`
Prueba	`<Clase>Test` en el mismo paquete, en `src/test/java`	`DispatchServiceTest`

Reglas de código obligatorias:

- **Inyección por constructor.** Nadie hace `new JdbcXxxDAO(...)` fuera de `AppContext`.
- **SQL siempre con `PreparedStatement`**, nunca concatenando valores.
- **Sin textos de usuario en la lógica:** los mensajes en español van en una clase de mensajes de `util`.
- **Tiempo con `Clock` inyectado** (`LocalDateTime.now(clock)`), así las pruebas y el simulador controlan la hora.
- **Los datos simulados se marcan como `SIMULACION`** (`TrafficSource`, `PositionSource`, `IncidentOrigin`).

5.10 Cómo agregar una funcionalidad

Ejemplo: *consultar los hospitales operativos*.

1. **Model** (Sherly): `model/Hospital.java` y, si hace falta, un enum en `model.enums`.
2. **DAO** (Andrés): interfaz `dao/HospitalDAO.java` con **solo** los métodos que se necesitan (`findOperational()`) e implementación `dao/jdbc/JdbcHospitalDAO.java` que traduce `HOSPITAL` → `Hospital`.
3. **Service**: `service/HospitalService.java` recibe `HospitalDAO` por constructor y aplica las reglas.
4. **Controller**: `controller/HospitalController.java` recibe `HospitalService`.
5. **AppContext** (Andrés): crear y conectar las tres clases.
6. **Pruebas**: el servicio se prueba con un `HospitalDAO` falso (sin Oracle); el DAO, contra la base local.
7. Ejecutar `mvn test`. Si `ArchitectureTest` falla, alguna capa importó algo que no debía.

6. Diseño: diagrama de clases

El diagrama de clases está escrito en **Mermaid** dentro de `docs/class-diagram.md`, organizado por capas. Aquí se incluye renderizado. Las imágenes están en alta resolución: en Word se pueden ampliar con el zoom, y la versión navegable está en el repositorio.

- Los nombres de clases, atributos y métodos están en inglés (convención del código). Las tablas de Oracle conservan sus nombres en español; la equivalencia está en `docs/convenciones-base-datos.md`.
- Para que se pueda leer, el sistema se divide en 8 diagramas, uno por capa o módulo. La sección 0 muestra cómo se conectan.
- **Getters y setters:** cada atributo tiene su getter. Los setters existen solo donde el dato puede cambiar, y por eso no se repiten en el diagrama. Sí se muestran los métodos con comportamiento.
- **Referencias entre entidades:** una entidad guarda el **id** de otra (`emergencyTypeId : long`), no el objeto completo. Así cada DAO mapea una sola tabla y no se cargan cadenas de objetos desde Oracle. Los servicios combinan los datos cuando hace falta. La multiplicidad de cada relación sigue las llaves foráneas de la base de datos.
- La sección 9 indica qué clases ya están **programadas** y cuáles están **diseñadas** pero pendientes.

> **Sincronización con el código**
>
> La prueba `ClassDiagramSyncTest` falla si una clase del código no aparece en el diagrama. Así el diagrama no se queda desactualizado.

6.0 Vista general por capas

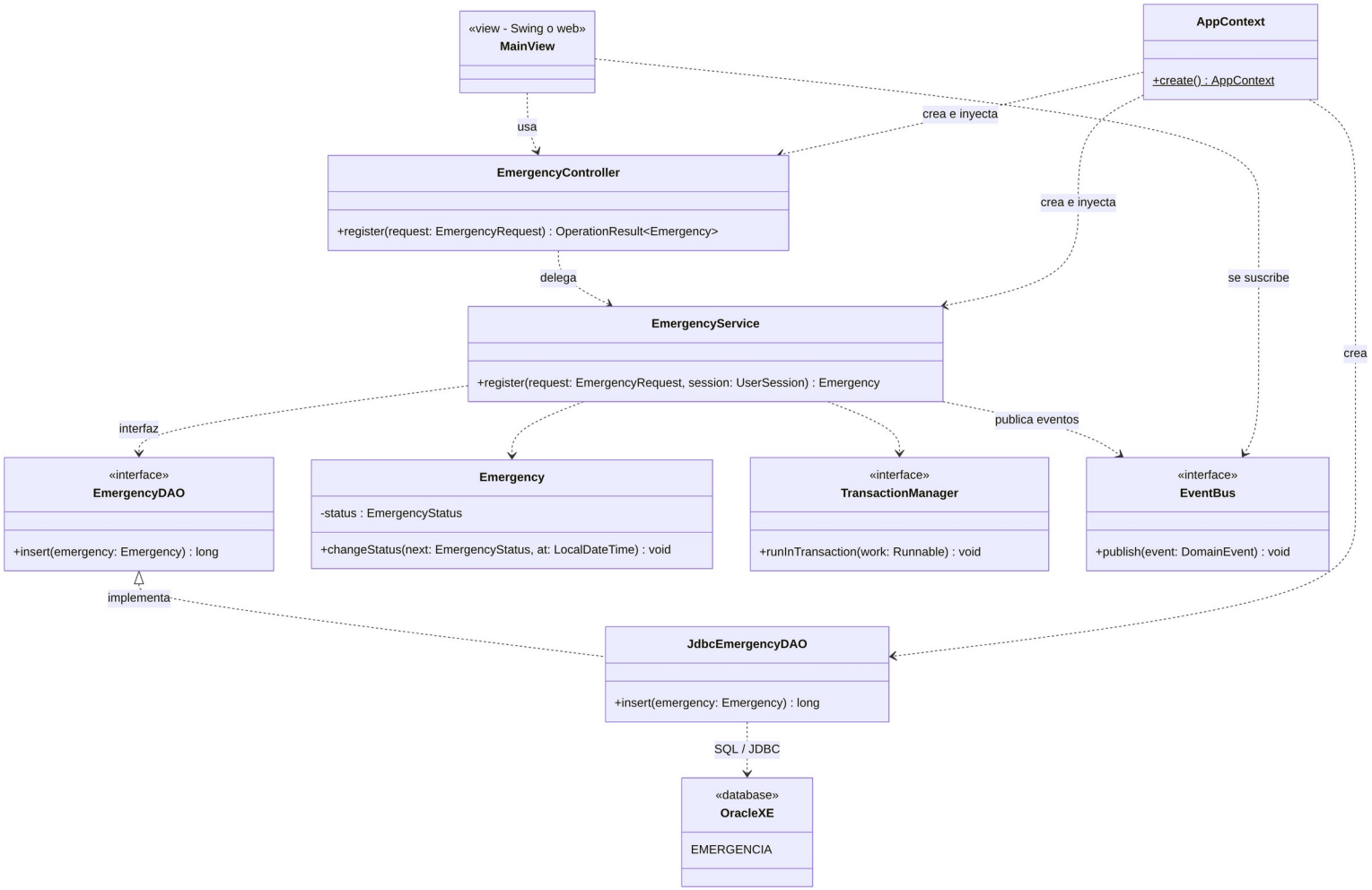


Figura 7. Vista general por capas

6.1.1 Tipos compartidos

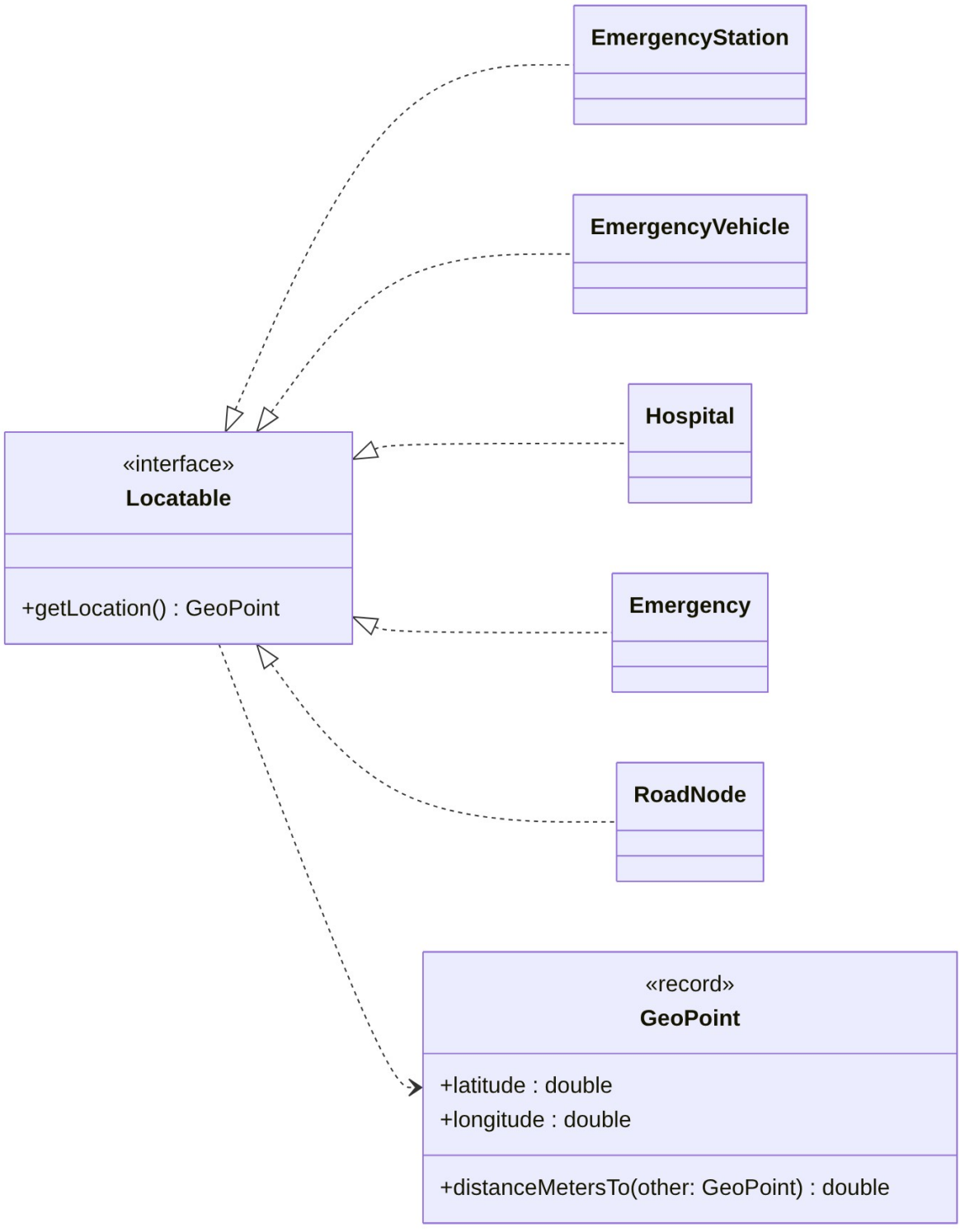


Figura 8. Tipos compartidos

6.1.2 Seguridad, alertas y auditoría

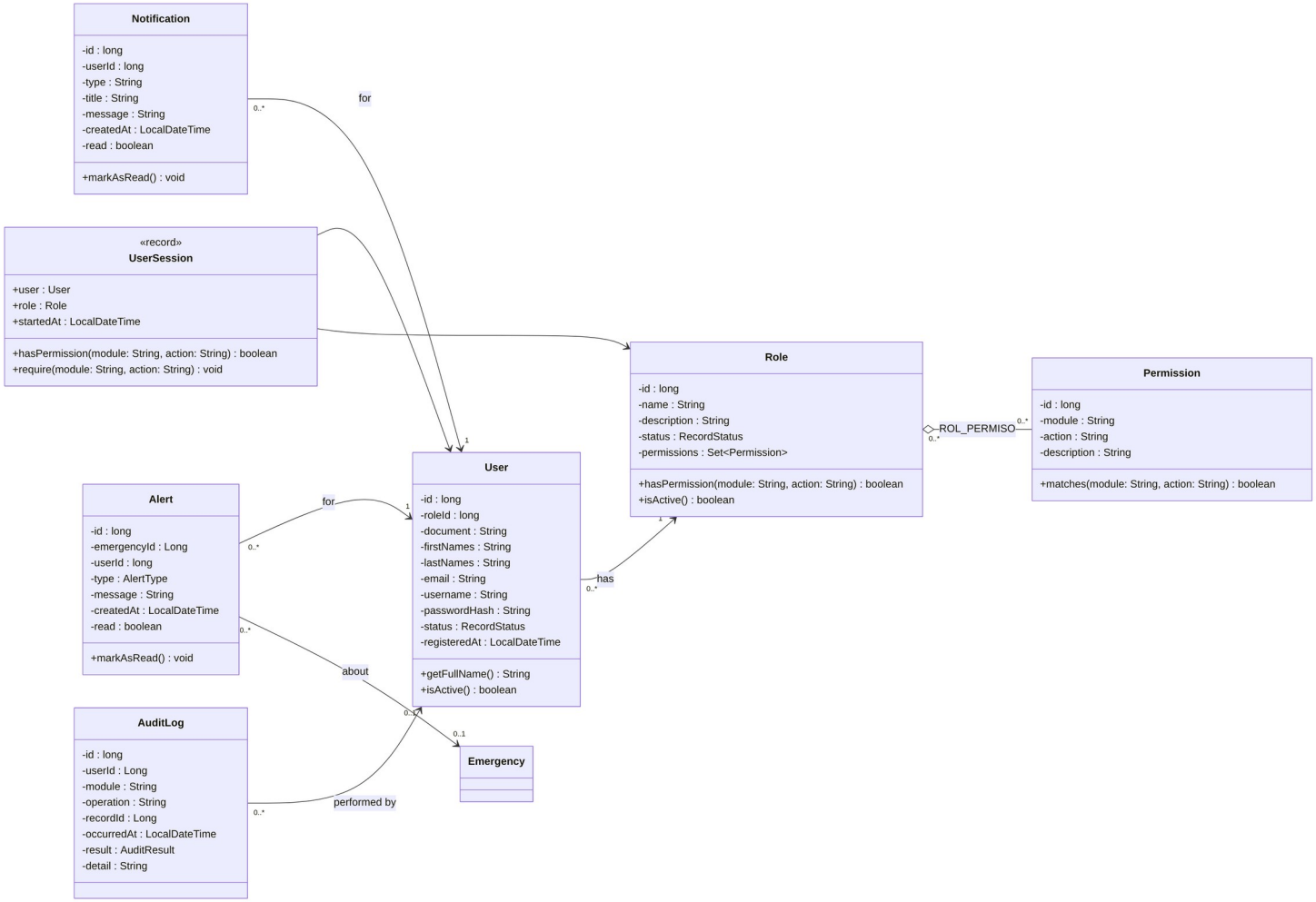


Figura 9. Seguridad, alertas y auditoría

6.1.3 Emergencias, unidades, hospitales y rutas

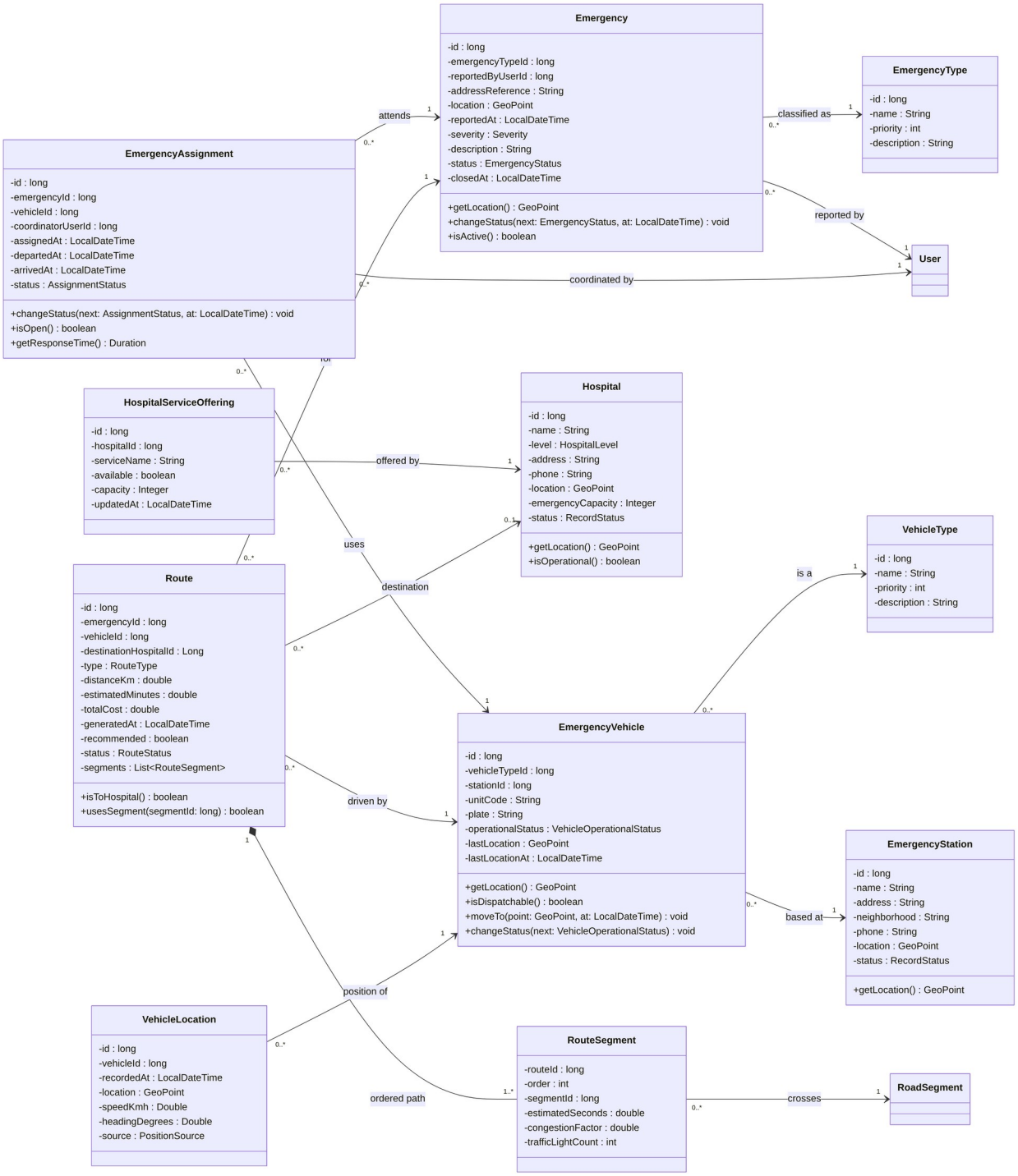


Figura 10. Emergencias, unidades, hospitales y rutas

6.1.4 Red vial y tráfico

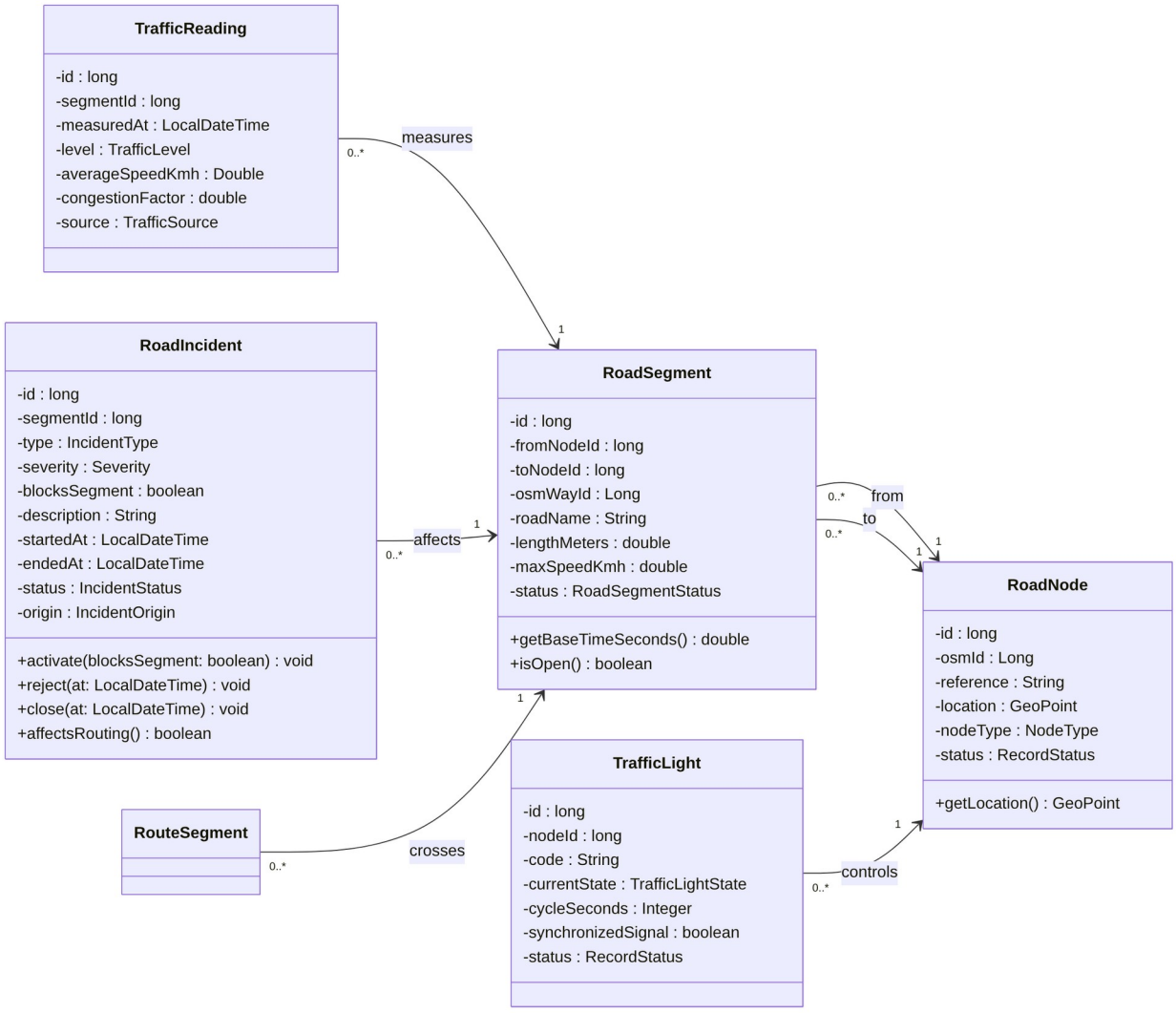


Figura 11. Red vial y tráfico

6.2.1 Estados con reglas de transición

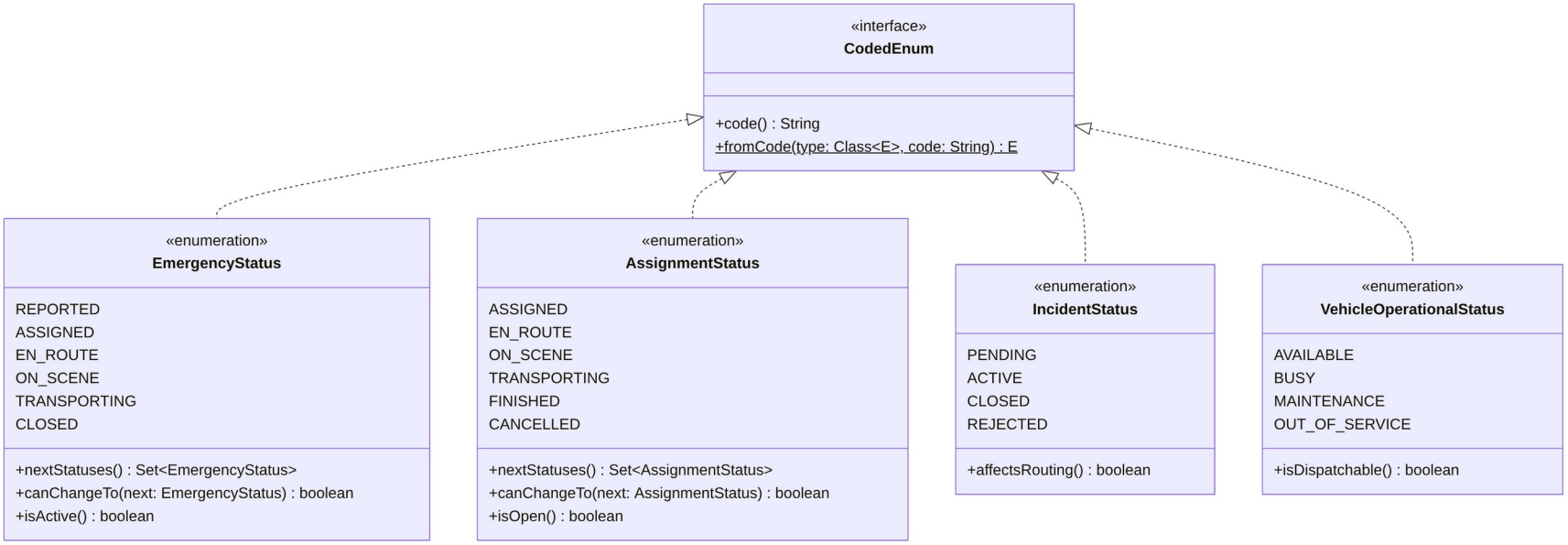


Figura 12. Estados con reglas de transición

6.2.2 Catálogos

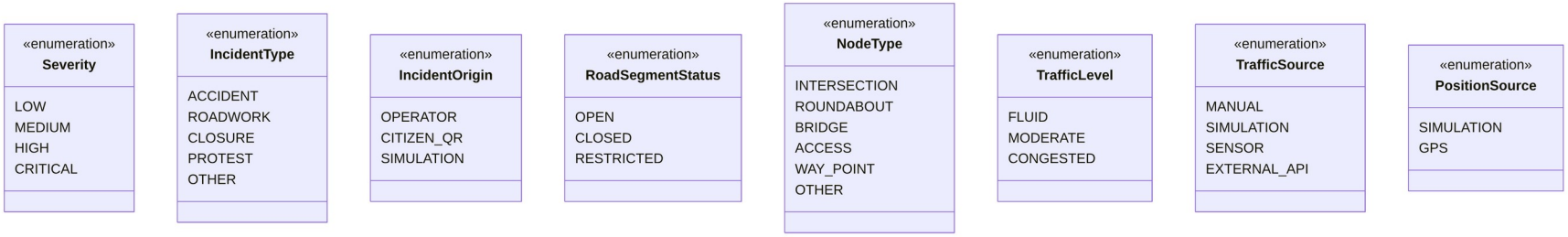


Figura 13. Catálogos

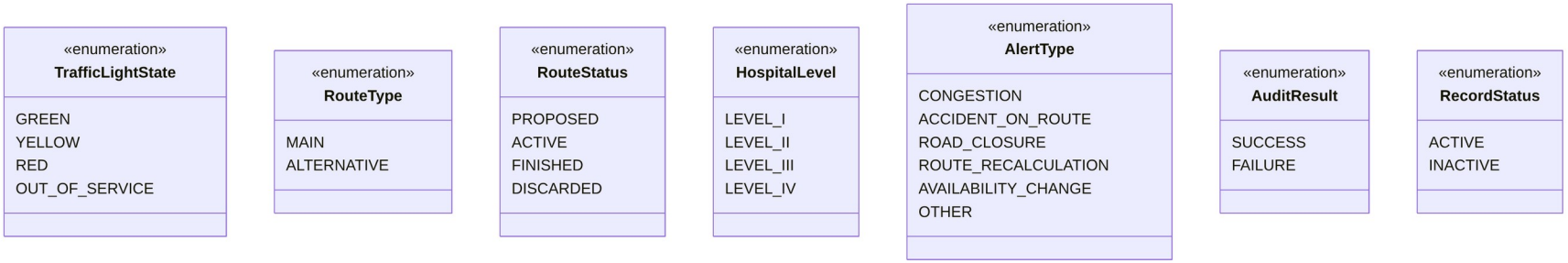


Figura 14. Catálogos

6.3 Capa DAO — interfaces

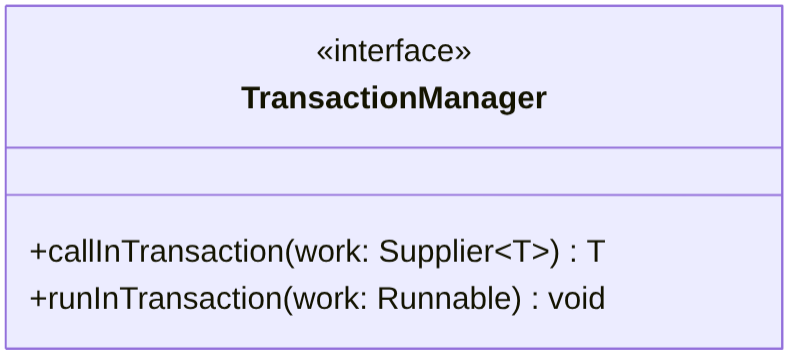


Figura 15. Capa DAO — interfaces

6.3.1 Seguridad, emergencias y despacho

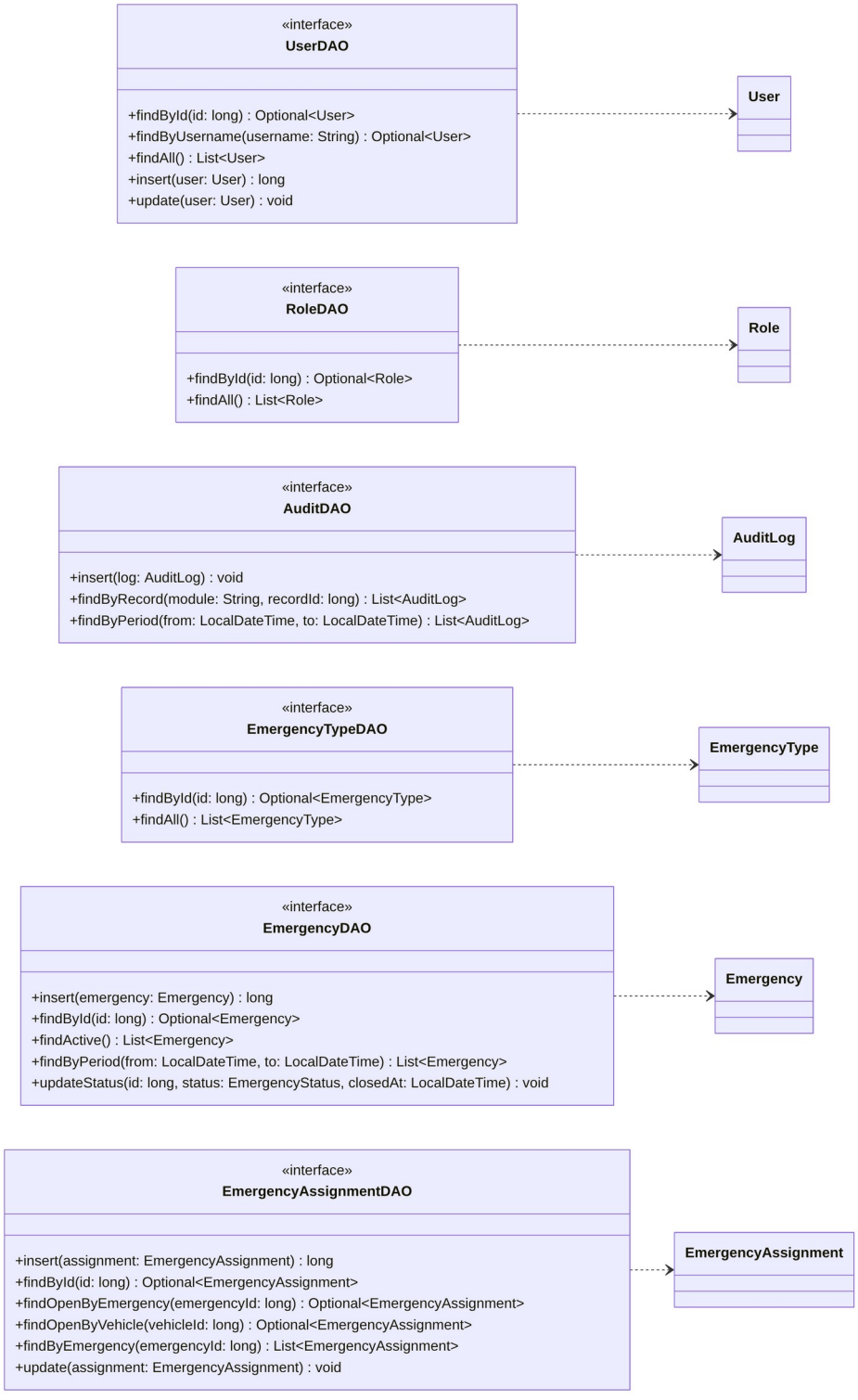


Figura 16. Seguridad, emergencias y despacho

6.3.2 Flota y hospitales

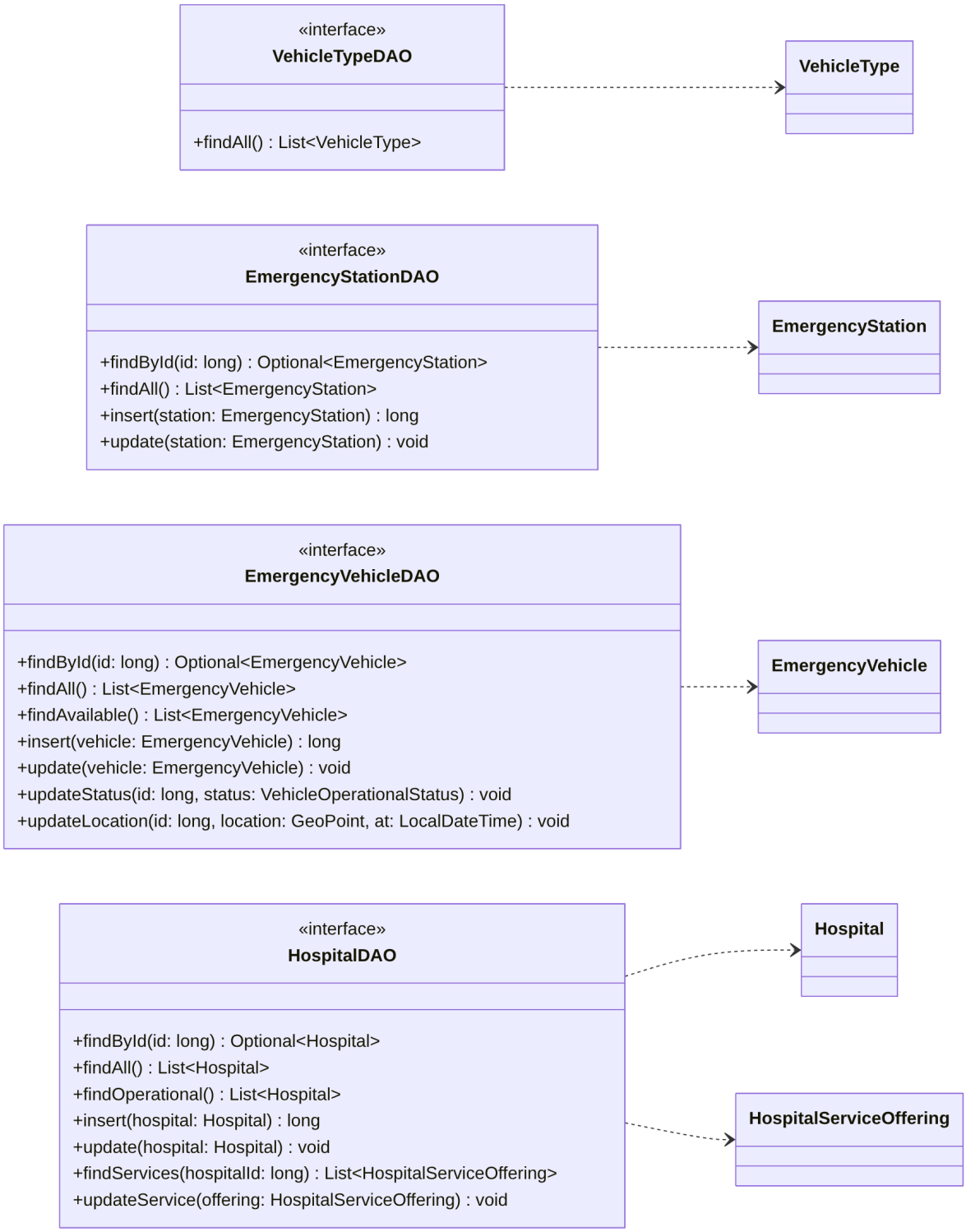


Figura 17. Flota y hospitales

6.3.3 Red vial, tráfico e incidentes

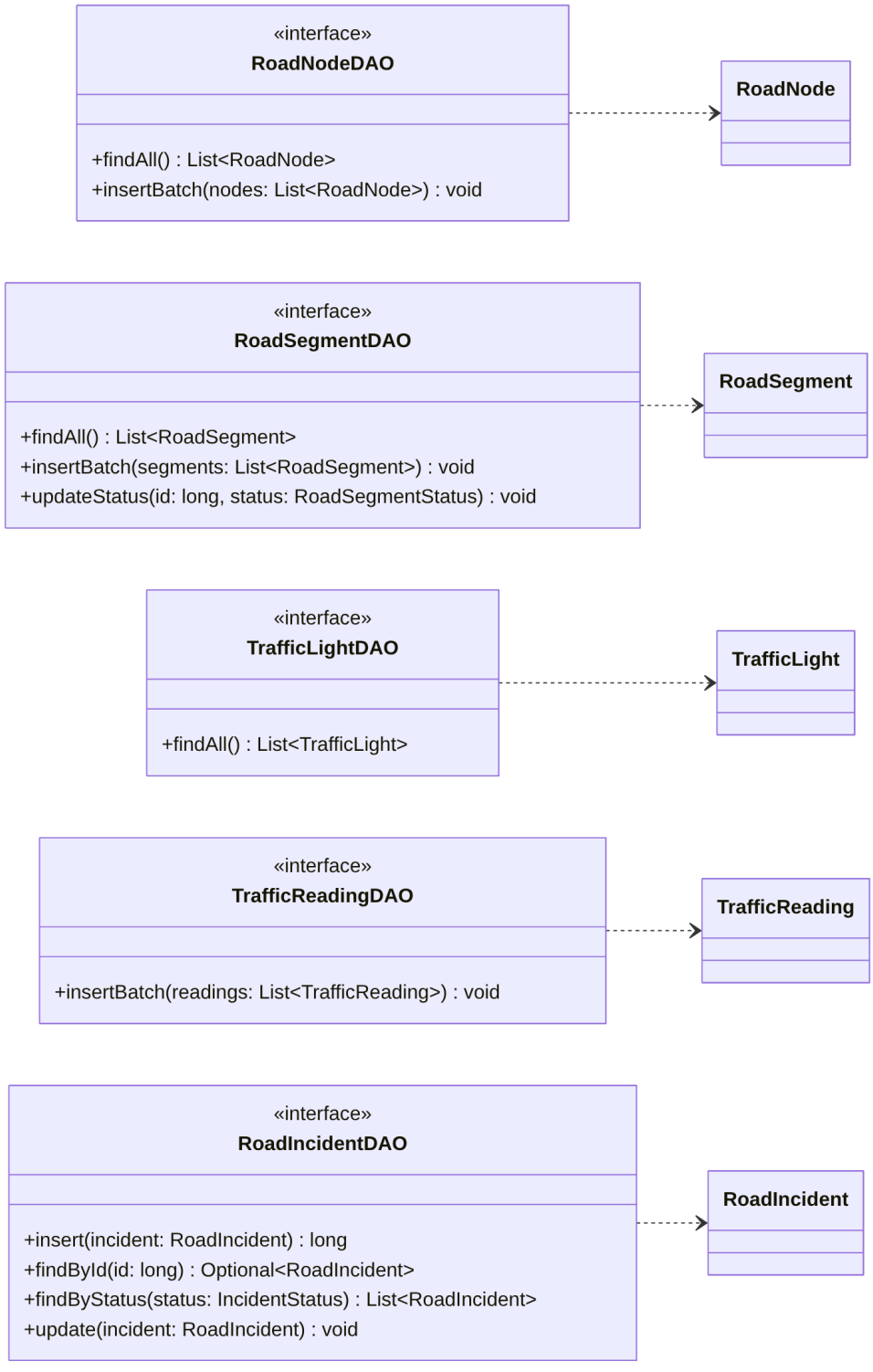


Figura 18. Red vial, tráfico e incidentes

6.3.4 Rutas, seguimiento y alertas

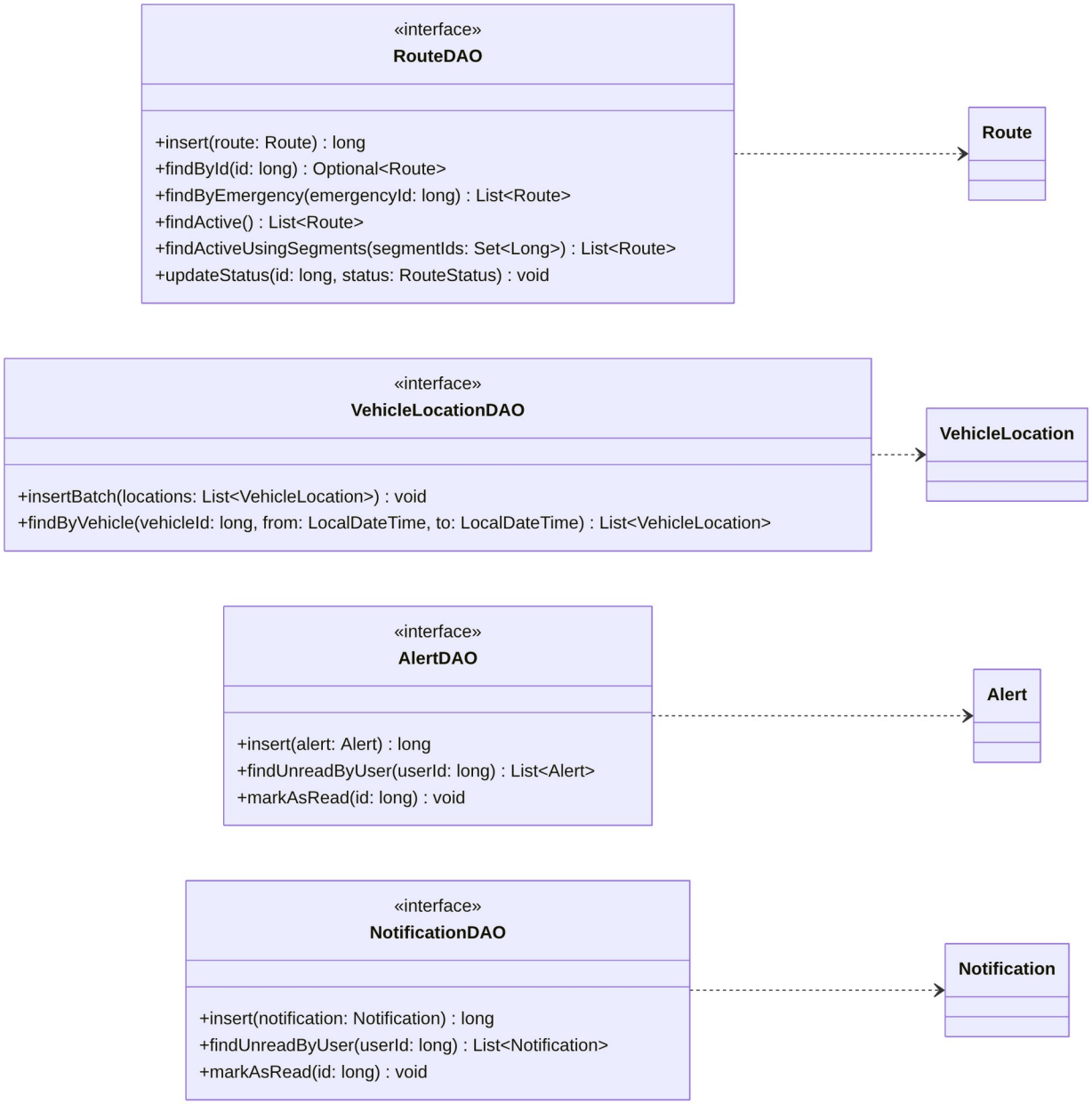


Figura 19. Rutas, seguimiento y alertas

6.4 Capa DAO — implementación JDBC (Oracle)

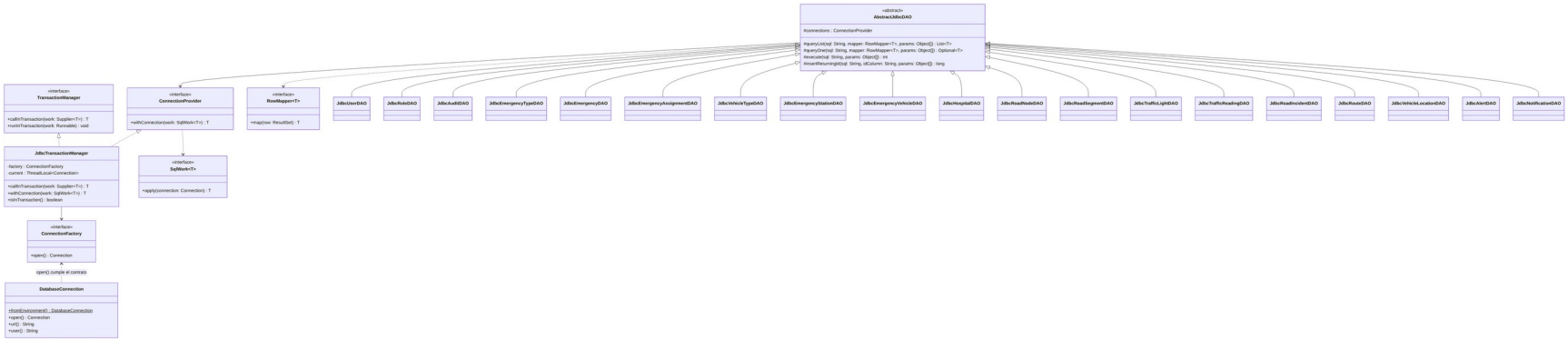


Figura 20. Capa DAO — implementación JDBC (Oracle)

6.5.1 Servicios

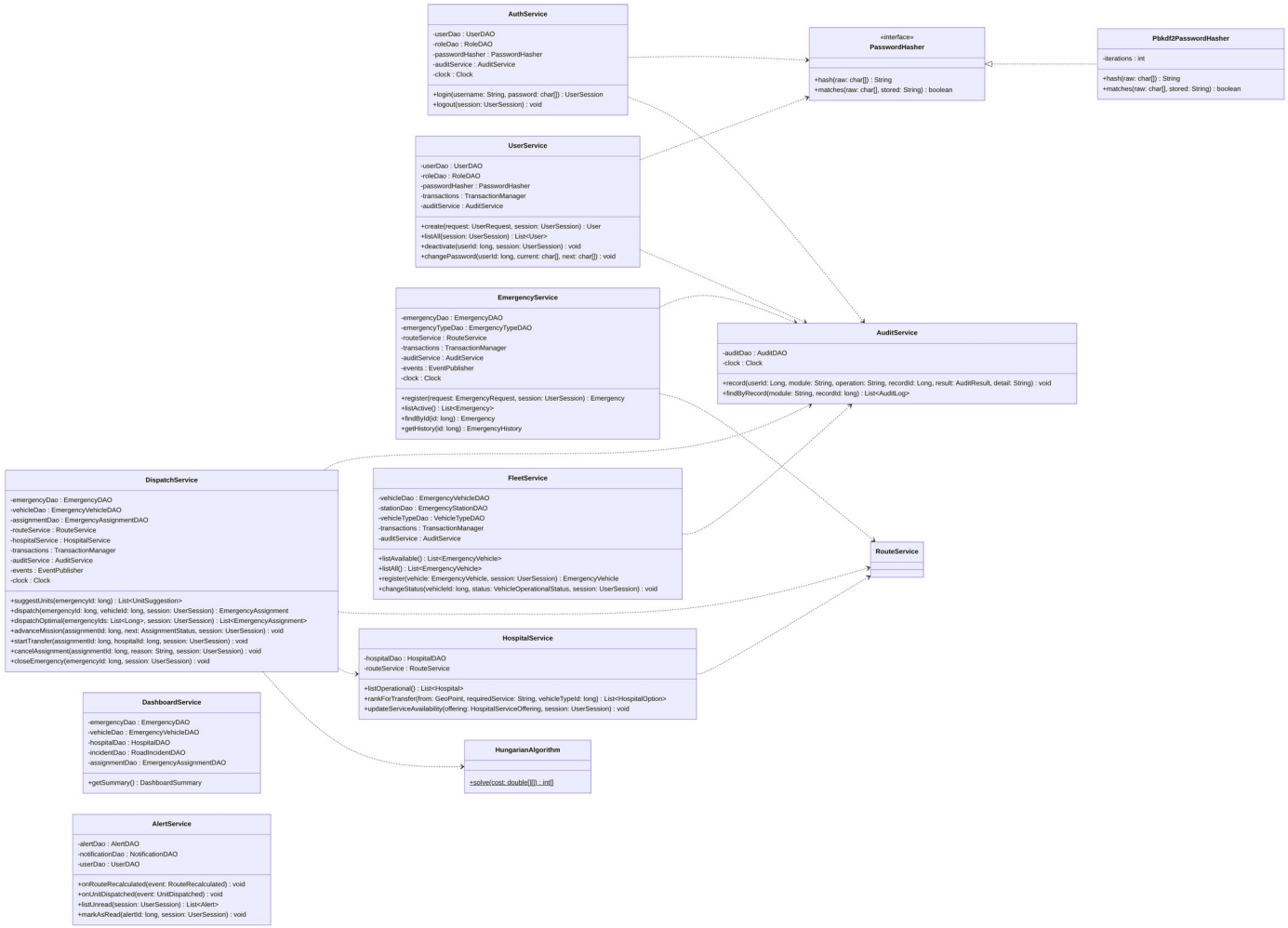


Figura 21. Servicios

6.5.2 DTO y utilidades

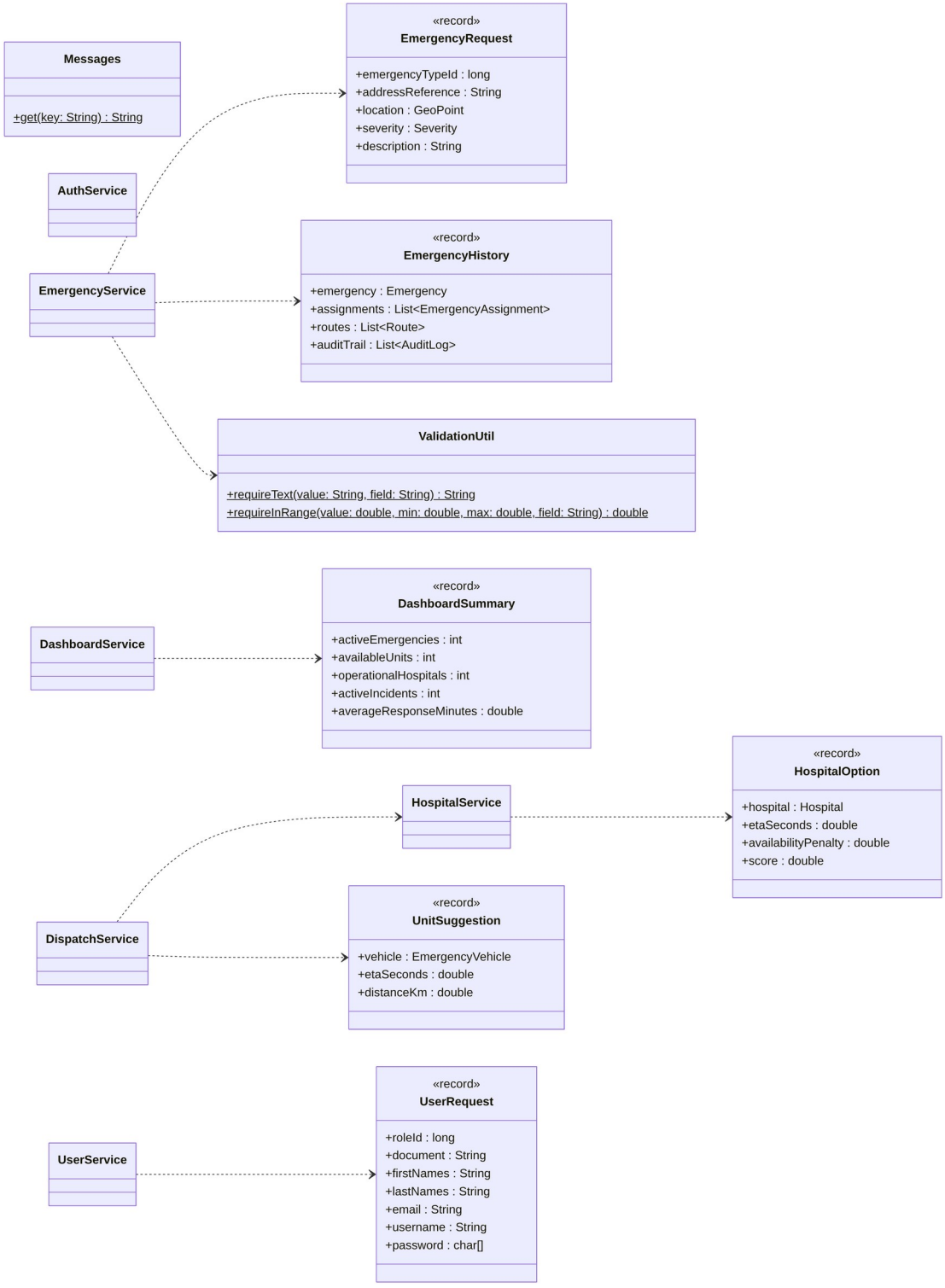


Figura 22. DTO y utilidades

6.6.1 Grafo y algoritmo de rutas (paquete graph, Kendrick)

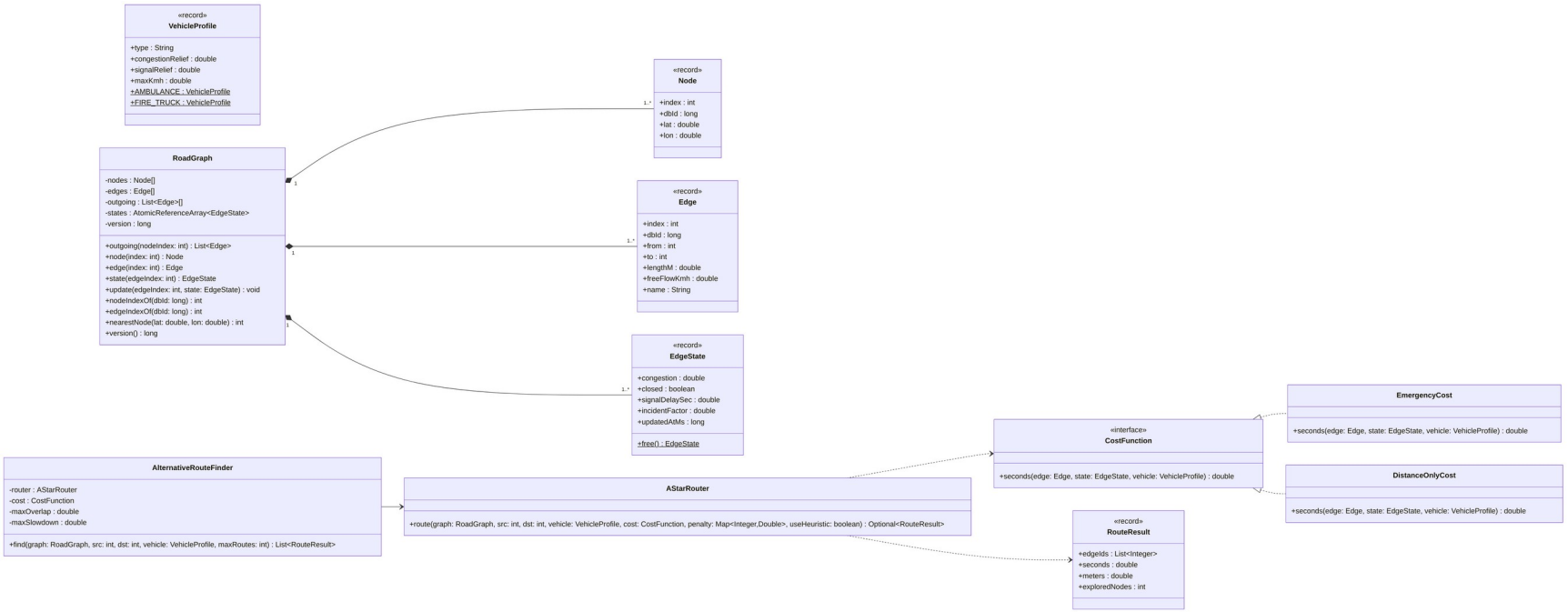


Figura 23. Grafo y algoritmo de rutas (paquete graph, Kendrick)

6.6.2 Rutas, tráfico, incidentes y recálculo (paquetes service y traffic)

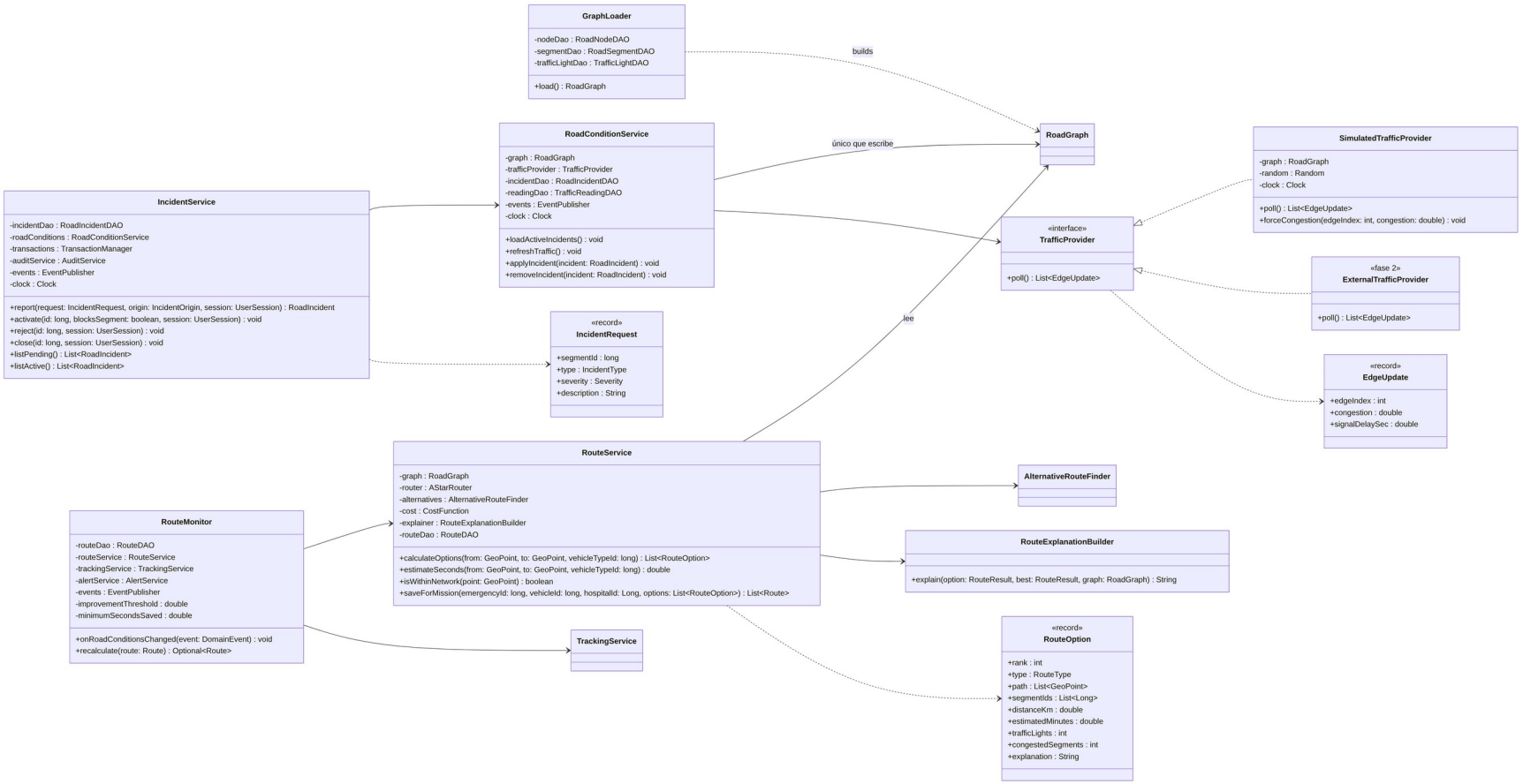


Figura 24. Rutas, tráfico, incidentes y recálculo (paquetes service y traffic)

6.6.3 Simulación y seguimiento (paquetes simulation y service)

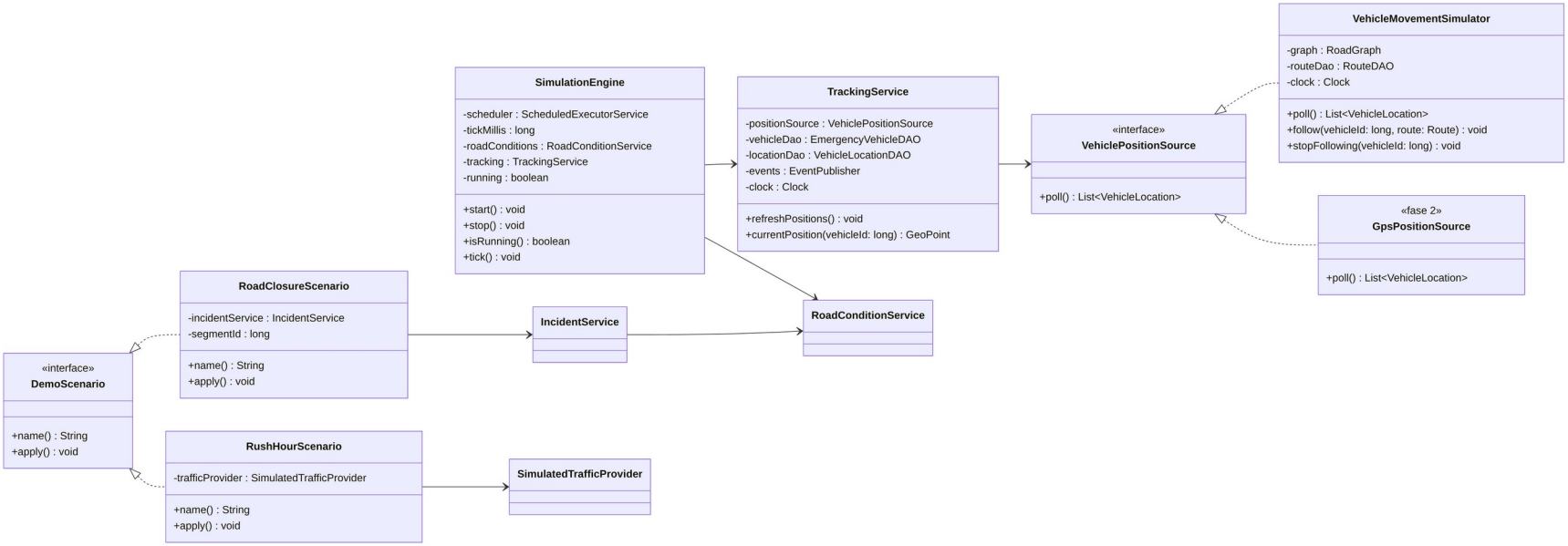


Figura 25. Simulación y seguimiento (paquetes simulation y service)

6.7 Eventos de tiempo real y excepciones



Figura 26. Eventos de tiempo real y excepciones

6.8.1 Controladores

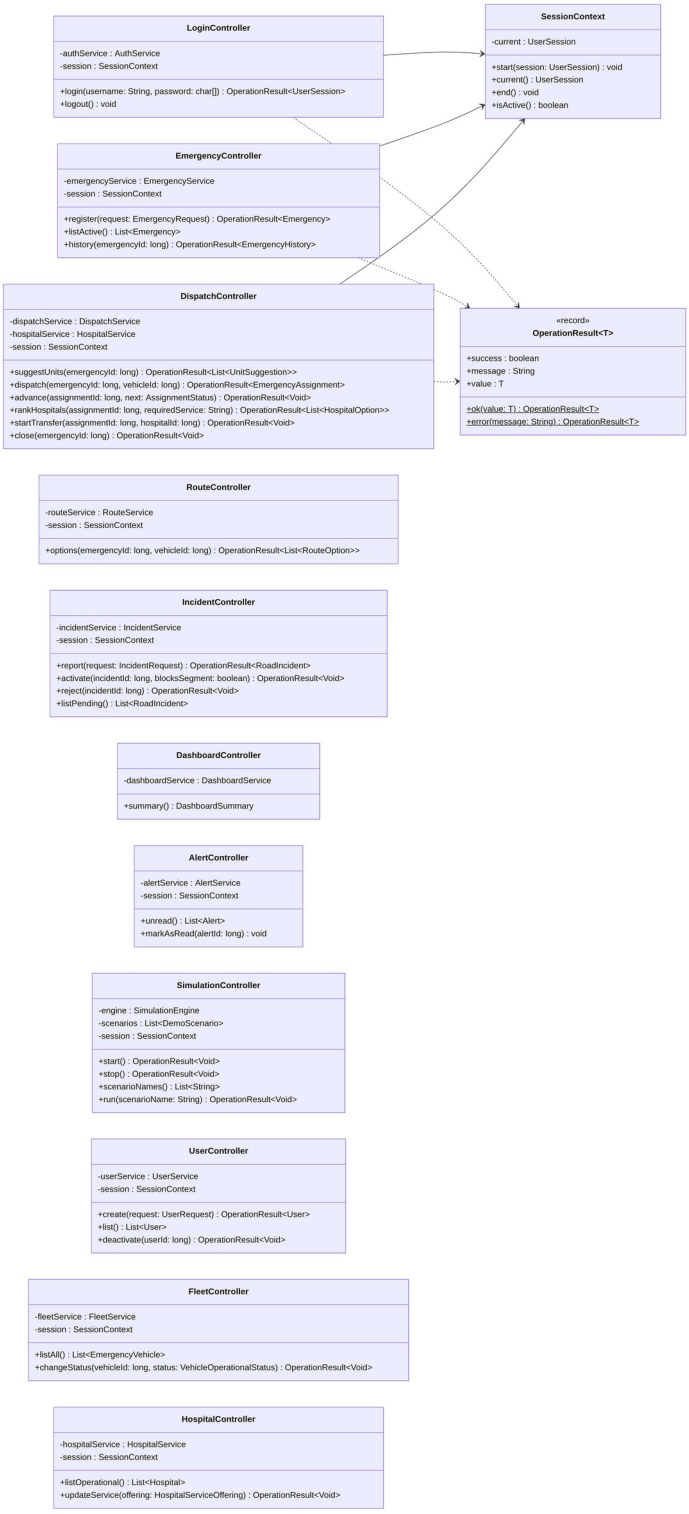


Figura 27. Controladores

6.8.2 Vista y arranque

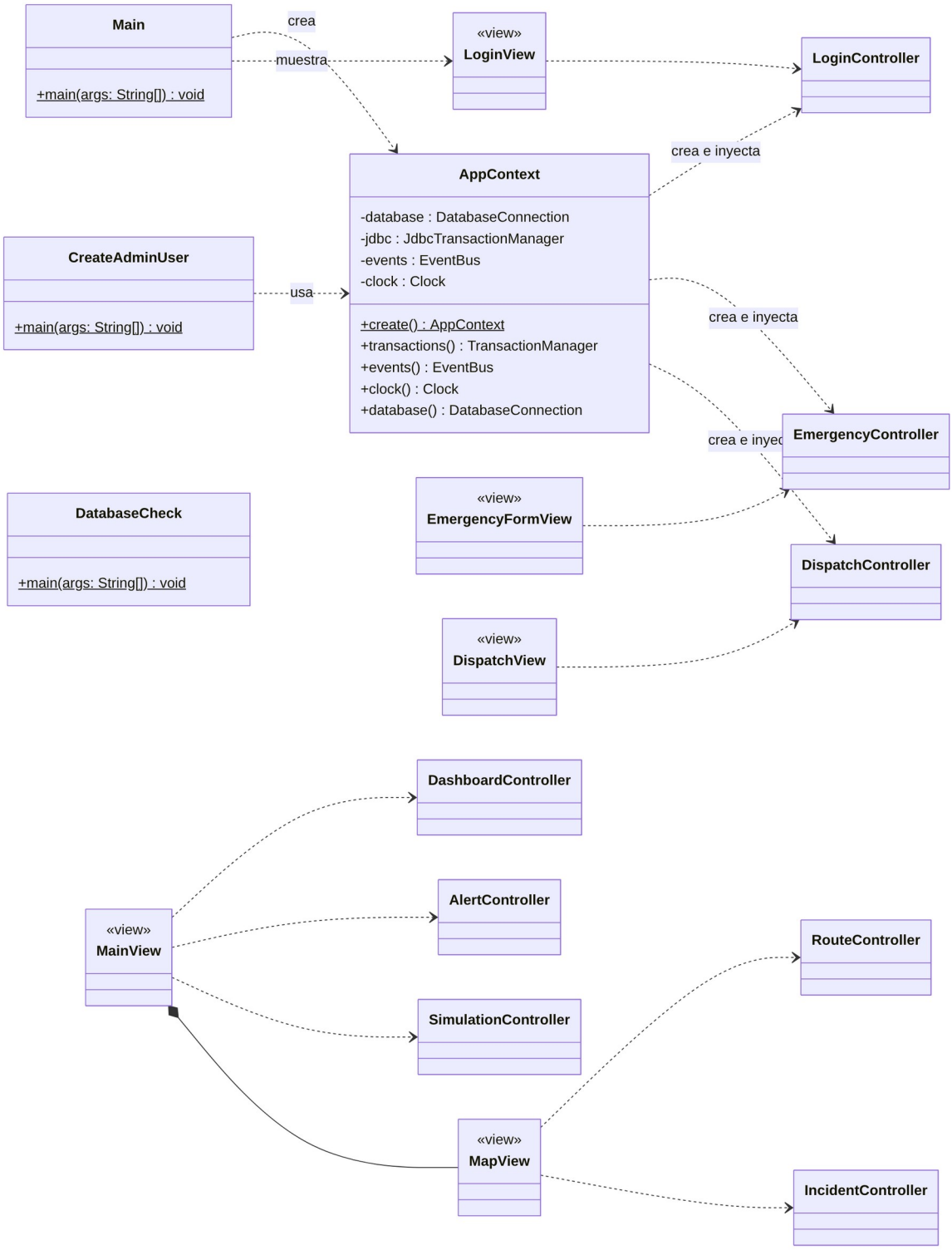


Figura 28. Vista y arranque

6.9 Eventos de tiempo real

Evento	Lo publica	Lo escuchan
`EmergencyRegistered`	`EmergencyService`	Vista (mapa y lista)
`UnitDispatched`	`DispatchService`	Vista, `AlertService`, `VehicleMovementSimulator`
`EmergencyStatusChanged`	`DispatchService`	Vista
`TrafficChanged`	`RoadConditionService`	`RouteMonitor`, vista (tramos congestionados)
`IncidentActivated`	`IncidentService`	`RouteMonitor`, vista
`VehiclePositionUpdated`	`TrackingService`	Vista (mueve el ícono de la unidad)
`RouteRecalculated`	`RouteMonitor`	Vista (nueva ruta), `AlertService`

6.10 Estado de implementación de las clases

Clase o grupo	Paquete	Estado
`CodedEnum` y las 19 enumeraciones	`model.enums`	**Programado y probado**
`TransactionManager`, `ConnectionProvider`, `ConnectionFactory`, `JdbcTransactionManager`	`dao`, `dao.jdbc`	**Programado y probado**
`DomainEvent`, `EventPublisher`, `EventBus`, `InMemoryEventBus`	`event`	**Programado y probado**
`RutaVidaException`, `ValidationException`, `NotFoundException`, `BusinessRuleException`, `DataAccessException`	`exception`	**Programado**
`DatabaseConnection`, `AppContext`, `DatabaseCheck`	`util`, `app`	**Programado y probado** contra Oracle XE 18c
`AccessDeniedException`, `RowMapper`, `AbstractJdbcDAO`, `CreateAdminUser`	`exception`, `dao.jdbc`, `app`	Diseñado
Entidades (sección 1), `GeoPoint`, `Locatable`, `UserSession`	`model`	Diseñado
Interfaces DAO y `JdbcXxxDAO`	`dao`, `dao.jdbc`	Diseñado
Servicios, DTO, `PasswordHasher`	`service`, `dto`, `util`	Diseñado
Motor de rutas, tráfico y simulación	`graph`, `traffic`, `simulation`, `service`	Diseñado (código de referencia en el documento consolidado, Parte IV)
Eventos concretos (`UnitDispatched`…)	`event`	Diseñado
Controladores, `SessionContext`, `OperationResult`	`controller`, `dto`	Diseñado
Vista	`view`	Pendiente de la decisión Swing o web

7. Base de datos

7.1 Motor y conexión

Concepto	Valor
Motor	Oracle Database XE (instalación local de cada integrante)
Servicio (PDB)	`XEPDB1`
URL JDBC	`jdbc:oracle:thin:@//localhost:1521/XEPDB1`
Usuario / esquema	`RUTAVIDA`
Contraseña	La define cada integrante; va en `backend/db.properties`, que **no** se sube a GitHub

Decisión del equipo (14/10/2026): se trabaja con Oracle local en lugar de Oracle Cloud.

Consecuencias que hay que tener presentes:

- Cada integrante tiene **su propia** base de datos. Los datos no se comparten: lo que uno inserta no lo ve otro. Por eso el esquema y los datos iniciales viven en scripts versionados en `database/`, y cualquier cambio de estructura se hace en un script nuevo, nunca a mano.
- La propuesta entregada al profesor dice "Oracle alojado en la nube". Hay que informarle del cambio o confirmar que lo acepta.
- Si más adelante se vuelve a la nube, el código Java no cambia: solo cambian la URL y las credenciales en `db.properties`.

7.2 Convenciones generales

Regla	Ejemplo
Tablas y columnas en español (diccionario aprobado). El DAO las traduce al Model en inglés.	`TRAMO_VIAL` → `RoadSegment`
Valores de estado y catálogo: MAYÚSCULAS, sin tildes y con _	`EN_RUTA`, `CRITICA`, `FUERA_SERVICIO`
Banderas sí/no: `CHAR(1)` con `'S'` / `'N'`	`es_recomendada`, `leida`, `disponible`
Estado de registros maestros: `CHAR(1)` con `'A'` (activo) / `'I'` (inactivo)	`ROL.estado`, `HOSPITAL.estado`
Identificadores: secuencia `SEQ_<TABLA>` usada como `DEFAULT` de la columna	`id_rol DEFAULT seq_rol.NEXTVAL`
Fechas: `TIMESTAMP` con `DEFAULT SYSTIMESTAMP` cuando es la fecha de creación	`fecha_hora_reporte`
Nombres de restricciones	`pk_`, `fk_`, `uq_`, `ck_`; índices `ix_` y `ux_` (únicos)
Coordenadas: `NUMBER(9,6)` con `CHECK` de rango	latitud entre -90 y 90

7.3 Catálogo de tablas

El modelo conserva las **23 tablas** del diseño original. Tres grupos: operación de emergencias, red vial (grafo) y control del sistema.

#	Tabla	Área	Propósito	Clase Java
1	ROL	Seguridad	Roles del sistema. Cada usuario tiene un rol principal.	Role
2	PERMISO	Seguridad	Catálogo de permisos: un módulo y una acción (CONSULTAR o GESTIONAR).	Permission
3	ROL_PERMISO	Seguridad	Relación muchos a muchos entre roles y permisos.	(Role.permissions)
4	USUARIO	Seguridad	Personas que operan el sistema.	User
5	ESTACION_EMERGENCIA	Operación	Estaciones de bomberos y bases de ambulancias.	EmergencyStation
6	TIPO_VEHICULO	Operación	Catálogo de tipos de unidad.	VehicleType
7	VEHICULO_EMERGENCIA	Operación	Ambulancias y vehículos de bomberos, con su última posición conocida.	EmergencyVehicle
8	HOSPITAL	Salud	Destinos hospitalarios de Valledupar.	Hospital
9	SERVICIO_HOSPITAL	Salud	Servicios que ofrece cada hospital y su disponibilidad.	HospitalServiceOffering
10	TIPO_EMERGENCIA	Emergencias	Clasificación de las emergencias.	EmergencyType
11	EMERGENCIA	Emergencias	Registro central de cada emergencia.	Emergency
12	NODO_VIAL	Red vial	Vértices del grafo vial: intersecciones y puntos de la vía.	RoadNode
13	TRAMO_VIAL	Red vial	Aristas dirigidas del grafo: una fila por sentido de circulación.	RoadSegment
14	SEMAFORO	Movilidad	Semáforos asociados a intersecciones.	TrafficLight
15	TRAFICO	Movilidad	Historial de mediciones de tráfico por tramo (se guarda por lotes).	TrafficReading
16	INCIDENTE_VIAL	Movilidad	Accidentes, obras y cierres que afectan tramos.	RoadIncident
17	RUTA	Rutas	Rutas calculadas: la principal y las alternativas son filas de esta tabla.	Route
18	RUTA_TRAMO	Rutas	Secuencia ordenada de tramos de una ruta.	RouteSegment
19	ASIGNACION_EMERGENCIA	Despacho	Unidad asignada a una emergencia y tiempos del servicio.	EmergencyAssignment
20	UBICACION_VEHICULO	Seguimiento	Historial de posiciones de las unidades (se guarda por lotes).	VehicleLocation
21	ALERTA	Notificaciones	Alertas operativas para los usuarios.	Alert
22	NOTIFICACION	Notificaciones	Mensajes internos para los usuarios.	Notification
23	AUDITORIA	Seguridad	Trazabilidad de operaciones críticas y de inicios de sesión.	AuditLog

7.4 Diccionario de datos

Generado directamente del script `database/01_esquema.sql` que se ejecutó en Oracle. **Nulo = No** indica columna obligatoria. Todos los identificadores usan una secuencia `SEQ_<TABLA>` como valor por defecto.

7.4.1 ROL

Roles del sistema. Cada usuario tiene un rol principal. Clase Java: `Role`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_rol`	NUMBER(10)	No	PK Secuencia SEQ_ROL	Identificador del rol
`nombre_rol`	VARCHAR2(50)	No	UNIQUE	Nombre del rol en mayúsculas (ADMINISTRADOR, OPERADOR_CENTRAL…)
`descripcion`	VARCHAR2(200)	Sí	—	Descripción del rol
`estado`	CHAR(1)	No	Valores: A, I DEFAULT 'A'	A = activo, I = inactivo

Restricciones de la tabla:

- `ck_rol_nombre` (CHECK): `nombre_rol = UPPER(nombre_rol)`

7.4.2 PERMISO

Catálogo de permisos: un módulo y una acción (CONSULTAR o GESTIONAR). Clase Java: `Permission`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_permiso`	NUMBER(10)	No	PK Secuencia SEQ_PERMISO	Identificador del permiso
`modulo`	VARCHAR2(60)	No	—	Módulo al que aplica (EMERGENCIAS, RUTAS…)
`accion`	VARCHAR2(40)	No	—	Acción permitida (CONSULTAR, GESTIONAR)
`descripcion`	VARCHAR2(200)	Sí	—	Detalle del permiso

Restricciones de la tabla:

- `uq_permiso_modulo_accion` (UNIQUE): `modulo, accion`
- `ck_permiso_mayusculas` (CHECK): `modulo = UPPER(modulo) AND accion = UPPER(accion)`

7.4.3 ROL_PERMISO

Relación muchos a muchos entre roles y permisos. Clase Java: `(Role.permissions)`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_rol`	NUMBER(10)	No	PK FK → ROL (cascada)	Rol autorizado

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_permiso	NUMBER(10)	No	PK FK → PERMISO (cascada)	Permiso asignado al rol

7.4.4 USUARIO

Personas que operan el sistema. Clase Java: `User`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_usuario	NUMBER(10)	No	PK Secuencia SEQ_USUARIO	Identificador del usuario
id_rol	NUMBER(10)	No	FK → ROL	Rol del usuario
documento	VARCHAR2(20)	No	UNIQUE	Documento de identidad
nombres	VARCHAR2(80)	No	—	Nombres
apellidos	VARCHAR2(80)	No	—	Apellidos
email	VARCHAR2(120)	No	UNIQUE	Correo electrónico
username	VARCHAR2(40)	No	UNIQUE	Nombre de usuario para iniciar sesión
password_hash	VARCHAR2(255)	No	—	Contraseña con hash PBKDF2 y sal; nunca en texto plano
estado	CHAR(1)	No	Valores: A, I DEFAULT 'A'	A = activo, I = inactivo
fecha_registro	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Fecha y hora de creación del usuario

Restricciones de la tabla:

- `ck_usuario_email` (CHECK): `email LIKE '%_@_%._%'`

7.4.5 ESTACION_EMERGENCIA

Estaciones de bomberos y bases de ambulancias. Clase Java: `EmergencyStation`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_estacion	NUMBER(10)	No	PK Secuencia SEQ_ESTACION_EMERGENCIA	Identificador de la estación
nombre	VARCHAR2(100)	No	—	Nombre de la estación o base
direccion	VARCHAR2(180)	No	—	Dirección
barrio	VARCHAR2(100)	Sí	—	Barrio o sector
telefono	VARCHAR2(20)	Sí	—	Teléfono de contacto
latitud	NUMBER(9,6)	No	—	Latitud (grados decimales)
longitud	NUMBER(9,6)	No	—	Longitud (grados decimales)
estado	CHAR(1)	No	Valores: A, I DEFAULT 'A'	A = activa, I = inactiva

Restricciones de la tabla:

- `ck_estacion_latitud` (CHECK): `latitud BETWEEN -90 AND 90`
- `ck_estacion_longitud` (CHECK): `longitud BETWEEN -180 AND 180`

7.4.6 TIPO_VEHICULO

Catálogo de tipos de unidad. Clase Java: `VehicleType`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_tipo_vehiculo`	NUMBER(10)	No	PK Secuencia SEQ_TIPO_VEHICULO	Identificador del tipo
`nombre`	VARCHAR2(50)	No	UNIQUE	AMBULANCIA, BOMBERO…
`prioridad`	NUMBER(2)	No	—	Prioridad operacional: 1 = más baja, 10 = más alta
`descripcion`	VARCHAR2(200)	Sí	—	Descripción

Restricciones de la tabla:

- `ck_tipo_vehiculo_nombre` (CHECK): `nombre = UPPER(nombre)`
- `ck_tipo_vehiculo_prior` (CHECK): `prioridad BETWEEN 1 AND 10`

7.4.7 VEHICULO_EMERGENCIA

Ambulancias y vehículos de bomberos, con su última posición conocida. Clase Java: `EmergencyVehicle`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_vehiculo`	NUMBER(10)	No	PK Secuencia SEQ_VEHICULO_EMERGENCIA	Identificador de la unidad
`id_tipo_vehiculo`	NUMBER(10)	No	FK → TIPO_VEHICULO	Tipo de vehículo
`id_estacion`	NUMBER(10)	No	FK → ESTACION_EMERGENCIA	Estación de origen
`codigo_unidad`	VARCHAR2(30)	No	UNIQUE	Código interno de la unidad
`placa`	VARCHAR2(10)	Sí	UNIQUE	Placa del vehículo
`estado_operativo`	VARCHAR2(30)	No	Valores: DISPONIBLE, OCUPADO, MANTENIMIENTO, FUERA_SERVICIO DEFAULT 'DISPONIBLE'	Disponibilidad de la unidad. La fase de la misión está en ASIGNACION_EMERGENCIA
`latitud`	NUMBER(9,6)	Sí	—	Última latitud conocida
`longitud`	NUMBER(9,6)	Sí	—	Última longitud conocida
`fecha_actualizacion`	TIMESTAMP	Sí	—	Momento de la última posición

Restricciones de la tabla:

- `ck_vehiculo_latitud` (CHECK): `latitud BETWEEN -90 AND 90`

- `ck_vehiculo_longitud` (CHECK): `longitud BETWEEN -180 AND 180`
- `ck_vehiculo_posicion` (CHECK): `(latitud IS NULL AND longitud IS NULL) OR (latitud IS NOT NULL AND longitud IS NOT NULL)`

7.4.8 HOSPITAL

Destinos hospitalarios de Valledupar. Clase Java: `Hospital`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_hospital`	NUMBER(10)	No	PK Secuencia SEQ_HOSPITAL	Identificador del hospital
`nombre`	VARCHAR2(150)	No	—	Nombre
`nivel`	VARCHAR2(3)	No	Valores: I, II, III, IV	Nivel de atención (I a IV)
`direccion`	VARCHAR2(180)	No	—	Dirección
`telefono`	VARCHAR2(20)	Sí	—	Teléfono
`latitud`	NUMBER(9,6)	No	—	Latitud
`longitud`	NUMBER(9,6)	No	—	Longitud
`capacidad_urgencias`	NUMBER(5)	Sí	—	Capacidad registrada de urgencias
`estado`	CHAR(1)	No	Valores: A, I DEFAULT 'A'	A = operativo, I = no operativo

Restricciones de la tabla:

- `ck_hospital_latitud` (CHECK): `latitud BETWEEN -90 AND 90`
- `ck_hospital_longitud` (CHECK): `longitud BETWEEN -180 AND 180`
- `ck_hospital_capacidad` (CHECK): `capacidad_urgencias >= 0`

7.4.9 SERVICIO_HOSPITAL

Servicios que ofrece cada hospital y su disponibilidad. Clase Java: `HospitalServiceOffering`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_servicio`	NUMBER(10)	No	PK Secuencia SEQ_SERVICIO_HOSPITAL	Identificador del servicio
`id_hospital`	NUMBER(10)	No	FK → HOSPITAL	Hospital que lo ofrece
`nombre_servicio`	VARCHAR2(100)	No	—	URGENCIAS, CIRUGIA, TRAUMA, UCI…
`disponible`	CHAR(1)	No	Valores: S, N DEFAULT 'S'	S = disponible, N = no disponible
`capacidad`	NUMBER(5)	Sí	—	Capacidad estimada
`fecha_actualizacion`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Última actualización de la disponibilidad

Restricciones de la tabla:

- `uq_servicio_hosp_nombre` (UNIQUE): `id_hospital, nombre_servicio`

- `ck_servicio_nombre` (CHECK): `nombre_servicio = UPPER(nombre_servicio)`
- `ck_servicio_capacidad` (CHECK): `capacidad >= 0`

7.4.10 TIPO_EMERGENCIA

Clasificación de las emergencias. Clase Java: `EmergencyType`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_tipo_emergencia`	NUMBER(10)	No	PK Secuencia SEQ_TIPO_EMERGENCIA	Identificador del tipo
`nombre`	VARCHAR2(80)	No	UNIQUE	ACCIDENTE_VIAL, INCENDIO, EMERGENCIA_MEDICA, RESCATE
`prioridad`	NUMBER(2)	No	—	Prioridad: 1 = más baja, 10 = más alta
`descripcion`	VARCHAR2(200)	Sí	—	Descripción

Restricciones de la tabla:

- `ck_tipo_emergencia_nombre` (CHECK): `nombre = UPPER(nombre)`
- `ck_tipo_emergencia_prior` (CHECK): `prioridad BETWEEN 1 AND 10`

7.4.11 EMERGENCIA

Registro central de cada emergencia. Clase Java: `Emergency`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_emergencia`	NUMBER(10)	No	PK Secuencia SEQ_EMERGENCIA	Identificador de la emergencia
`id_tipo_emergencia`	NUMBER(10)	No	FK → TIPO_EMERGENCIA	Tipo de emergencia
`id_usuario_reporta`	NUMBER(10)	No	FK → USUARIO	Usuario que la registró
`direccion_referencia`	VARCHAR2(200)	No	—	Dirección o referencia textual del lugar
`latitud`	NUMBER(9,6)	No	—	Latitud del lugar
`longitud`	NUMBER(9,6)	No	—	Longitud del lugar
`fecha_hora_reporte`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento del reporte
`severidad`	VARCHAR2(20)	No	Valores: BAJA, MEDIA, ALTA, CRITICA	Severidad del evento
`descripcion`	VARCHAR2(400)	Sí	—	Detalle del evento
`estado`	VARCHAR2(30)	No	Valores: REPORTADA, ASIGNADA, EN_RUTA, EN_SITIO, TRASLADANDO, CERRADA DEFAULT 'REPORTADA'	Estado en el ciclo de atención
`fecha_hora_cierre`	TIMESTAMP	Sí	—	Momento del cierre; obligatorio si el estado es CERRADA

Restricciones de la tabla:

- `ck_emergencia_latitud` (CHECK): `latitud BETWEEN -90 AND 90`
- `ck_emergencia_longitud` (CHECK): `longitud BETWEEN -180 AND 180`
- `ck_emergencia_cierre` (CHECK): `(estado = 'CERRADA' AND fecha_hora_cierre IS NOT NULL) OR (estado <> 'CERRADA' AND fecha_hora_cierre IS NULL)`
- `ck_emergencia_fechas` (CHECK): `fecha_hora_cierre IS NULL OR fecha_hora_cierre >= fecha_hora_reporte`

7.4.12 NODO_VIAL

Vértices del grafo vial: intersecciones y puntos de la vía. Clase Java: `RoadNode`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_nodo`	NUMBER(10)	No	PK Secuencia SEQ_NODO_VIAL	Identificador del nodo
`osm_id`	NUMBER(19)	Sí	UNIQUE	Identificador del nodo en OpenStreetMap (para recargar sin duplicar)
`referencia`	VARCHAR2(120)	Sí	—	Intersección o punto de referencia
`latitud`	NUMBER(9,6)	No	—	Latitud
`longitud`	NUMBER(9,6)	No	—	Longitud
`tipo_nodo`	VARCHAR2(30)	No	Valores: INTERSECCION, GLORIETA, PUENTE, ACCESO, PUNTO_VIA, OTRO DEFAULT 'INTERSECCION'	Tipo de nodo
`estado`	CHAR(1)	No	Valores: A, I DEFAULT 'A'	A = activo, I = inactivo

Restricciones de la tabla:

- `ck_nodo_vial_latitud` (CHECK): `latitud BETWEEN -90 AND 90`
- `ck_nodo_vial_longitud` (CHECK): `longitud BETWEEN -180 AND 180`

7.4.13 TRAMO_VIAL

Aristas dirigidas del grafo: una fila por sentido de circulación. Clase Java: `RoadSegment`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_tramo`	NUMBER(10)	No	PK Secuencia SEQ_TRAMO_VIAL	Identificador del tramo
`id_nodo_origen`	NUMBER(10)	No	FK → NODO_VIAL	Nodo donde empieza el tramo
`id_nodo_destino`	NUMBER(10)	No	FK → NODO_VIAL	Nodo donde termina el tramo
`osm_way_id`	NUMBER(19)	Sí	—	Identificador de la vía en OpenStreetMap
`nombre_via`	VARCHAR2(120)	No	—	Calle, carrera o avenida
`distancia_m`	NUMBER(10,2)	No	—	Longitud del tramo en metros
`velocidad_max_kmh`	NUMBER(5,2)	No	—	Velocidad de referencia

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
tiempo_base_seg	NUMBER(10,2)	Sí	Columna virtual	Columna virtual: distancia_m / (velocidad_max_kmh / 3,6)
estado	VARCHAR2(30)	No	Valores: ABIERTO, CERRADO, RESTRINGIDO DEFAULT 'ABIERTO'	Estado administrativo de la vía

Restricciones de la tabla:

- ck_tramo_nodos (CHECK): id_nodo_origen <> id_nodo_destino
- ck_tramo_distancia (CHECK): distancia_m > 0
- ck_tramo_velocidad (CHECK): velocidad_max_kmh > 0 AND velocidad_max_kmh <= 120

7.4.14 SEMAFORO

Semáforos asociados a intersecciones. Clase Java: TrafficLight.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_semaforo	NUMBER(10)	No	PK Secuencia SEQ_SEMAFORO	Identificador del semáforo
id_nodo	NUMBER(10)	No	FK → NODO_VIAL	Intersección que controla
codigo	VARCHAR2(30)	No	UNIQUE	Código del semáforo
estado_actual	VARCHAR2(20)	No	Valores: VERDE, AMARILLO, ROJO, FUERA_SERVICIO DEFAULT 'VERDE'	Luz actual (simulada en la fase 1)
tiempo_ciclo_seg	NUMBER(4)	Sí	—	Duración aproximada del ciclo
sincronizado	CHAR(1)	No	Valores: S, N DEFAULT 'N'	S = sincronizado, N = no
estado	CHAR(1)	No	Valores: A, I DEFAULT 'A'	A = activo, I = inactivo

Restricciones de la tabla:

- ck_semaforo_ciclo (CHECK): tiempo_ciclo_seg > 0

7.4.15 TRAFICO

Historial de mediciones de tráfico por tramo (se guarda por lotes). Clase Java: TrafficReading.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_trafico	NUMBER(12)	No	PK Secuencia SEQ_TRAFICO	Identificador de la medición
id_tramo	NUMBER(10)	No	FK → TRAMO_VIAL	Tramo medido
fecha_hora	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento de la medición
nivel	VARCHAR2(20)	No	Valores: FLUIDO, MODERADO, CONGESTIONADO	Nivel de tráfico

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
velocidad_promedio_kmh	NUMBER(6,2)	Sí	—	Velocidad observada
factor_congestion	NUMBER(5,2)	No	—	1,00 = flujo libre; 2,00 = el doble de lento
fuente	VARCHAR2(40)	No	Valores: MANUAL, SIMULACION, SENSOR, API_EXTERNA DEFAULT 'SIMULACION'	Origen del dato; distingue simulado de real

Restricciones de la tabla:

- ck_trafico_velocidad (CHECK): velocidad_promedio_kmh >= 0
- ck_trafico_factor (CHECK): factor_congestion >= 1

7.4.16 INCIDENTE_VIAL

Accidentes, obras y cierres que afectan tramos. Clase Java: RoadIncident.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_incidente	NUMBER(10)	No	PK Secuencia SEQ_INCIDENTE_VIAL	Identificador del incidente
id_tramo	NUMBER(10)	No	FK → TRAMO_VIAL	Tramo afectado
tipo_incidente	VARCHAR2(60)	No	Valores: ACCIDENTE, OBRA, CIERRE, MANIFESTACION, OTRO	Tipo de incidente
severidad	VARCHAR2(20)	No	Valores: BAJA, MEDIA, ALTA, CRITICA	Severidad del incidente
bloquea_tramo	CHAR(1)	No	Valores: S, N DEFAULT 'N'	S = el motor trata el tramo como cerrado; N = solo lo penaliza
descripcion	VARCHAR2(300)	Sí	—	Detalle
fecha_inicio	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Inicio del incidente
fecha_fin	TIMESTAMP	Sí	—	Fin del incidente
estado	VARCHAR2(20)	No	Valores: PENDIENTE, ACTIVO, CERRADO, RECHAZADO DEFAULT 'PENDIENTE'	PENDIENTE por defecto; solo ACTIVO afecta las rutas
origen	VARCHAR2(20)	No	Valores: OPERADOR, QR_CIUDADANO, SIMULACION DEFAULT 'OPERADOR'	Quién lo reportó

Restricciones de la tabla:

- ck_incidente_fechas (CHECK): fecha_fin IS NULL OR fecha_fin >= fecha_inicio

7.4.17 RUTA

Rutas calculadas: la principal y las alternativas son filas de esta tabla. Clase Java: Route.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_ruta	NUMBER(12)	No	PK Secuencia SEQ_RUTA	Identificador de la ruta
id_emergencia	NUMBER(10)	No	FK → EMERGENCIA	Emergencia atendida
id_vehiculo	NUMBER(10)	No	FK → VEHICULO_EMERGENCIA	Unidad que la recorre
id_hospital_destino	NUMBER(10)	Sí	FK → HOSPITAL	Vacío: hacia la emergencia. Con valor: traslado al hospital
tipo_ruta	VARCHAR2(20)	No	Valores: PRINCIPAL, ALTERNATIVA	Principal o alternativa
distancia_km	NUMBER(8,2)	No	—	Distancia calculada
tiempo_estimado_min	NUMBER(8,2)	No	—	Tiempo estimado
costo_total	NUMBER(12,2)	No	—	Costo en el grafo ponderado
fecha_generacion	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento del cálculo
es_recomendada	CHAR(1)	No	Valores: S, N DEFAULT 'N'	S = recomendada por el sistema
estado	VARCHAR2(20)	No	Valores: PROPUESTA, ACTIVA, FINALIZADA, DESCARTADA DEFAULT 'PROPUESTA'	Estado de la ruta

Restricciones de la tabla:

- `ck_ruta_valores` (CHECK): `distancia_km >= 0 AND tiempo_estimado_min >= 0 AND costo_total >= 0`

7.4.18 RUTA_TRAMO

Secuencia ordenada de tramos de una ruta. Clase Java: `RouteSegment`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
id_ruta	NUMBER(12)	No	PK FK → RUTA (cascada)	Ruta
orden_tramo	NUMBER(5)	No	PK	Posición del tramo dentro de la ruta (1, 2, 3…)
id_tramo	NUMBER(10)	No	FK → TRAMO_VIAL	Tramo recorrido
tiempo_estimado_seg	NUMBER(10,2)	Sí	—	Tiempo estimado en ese tramo
factor_congestion	NUMBER(5,2)	Sí	—	Factor usado en el cálculo
semaforos_afectados	NUMBER(3)	Sí	—	Cantidad de semáforos en el tramo

Restricciones de la tabla:

- `ck_ruta_tramo_orden` (CHECK): `orden_tramo >= 1`
- `ck_ruta_tramo_tiempo` (CHECK): `tiempo_estimado_seg >= 0`
- `ck_ruta_tramo_factor` (CHECK): `factor_congestion >= 1`
- `ck_ruta_tramo_semaforos` (CHECK): `semaforos_afectados >= 0`

7.4.19 ASIGNACION_EMERGENCIA

Unidad asignada a una emergencia y tiempos del servicio. Clase Java: `EmergencyAssignment`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_asignacion`	NUMBER(10)	No	PK Secuencia SEQ_ASIGNACION_EMERGENCIA	Identificador de la asignación
`id_emergencia`	NUMBER(10)	No	FK → EMERGENCIA	Emergencia atendida
`id_vehiculo`	NUMBER(10)	No	FK → VEHICULO_EMERGENCIA	Unidad asignada
`id_usuario_coordinador`	NUMBER(10)	No	FK → USUARIO	Coordinador que hizo el despacho
`fecha_asignacion`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento del despacho
`hora_salida`	TIMESTAMP	Sí	—	Salida de la unidad (al pasar a EN_RUTA)
`hora_llegada`	TIMESTAMP	Sí	—	Llegada al sitio (al pasar a EN_SITIO)
`estado`	VARCHAR2(30)	No	Valores: ASIGNADA, EN_RUTA, EN_SITIO, TRASLADANDO, FINALIZADA, CANCELADA DEFAULT 'ASIGNADA'	Fase de la misión

Restricciones de la tabla:

- `ck_asignacion_salida` (CHECK): `hora_salida IS NULL OR hora_salida >= fecha_asignacion`
- `ck_asignacion_llegada` (CHECK): `hora_llegada IS NULL OR (hora_salida IS NOT NULL AND hora_llegada >= hora_salida)`

7.4.20 UBICACION_VEHICULO

Historial de posiciones de las unidades (se guarda por lotes). Clase Java: `VehicleLocation`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_ubicacion`	NUMBER(12)	No	PK Secuencia SEQ_UBICACION_VEHICULO	Identificador del registro
`id_vehiculo`	NUMBER(10)	No	FK → VEHICULO_EMERGENCIA	Unidad
`fecha_hora`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento de la posición
`latitud`	NUMBER(9,6)	No	—	Latitud
`longitud`	NUMBER(9,6)	No	—	Longitud
`velocidad_kmh`	NUMBER(7,2)	Sí	—	Velocidad
`direccion_grados`	NUMBER(6,2)	Sí	—	Rumbo (0 a 359,99°)
`fuente`	VARCHAR2(20)	No	Valores: SIMULACION, GPS DEFAULT 'SIMULACION'	SIMULACION o GPS; distingue simulado de real

Restricciones de la tabla:

- `ck_ubicacion_latitud` (CHECK): `latitud BETWEEN -90 AND 90`
- `ck_ubicacion_longitud` (CHECK): `longitud BETWEEN -180 AND 180`
- `ck_ubicacion_vel` (CHECK): `velocidad_kmh >= 0`
- `ck_ubicacion_rumbo` (CHECK): `direccion_grados >= 0 AND direccion_grados < 360`

7.4.21 ALERTA

Alertas operativas para los usuarios. Clase Java: `Alert`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_alerta`	NUMBER(12)	No	PK Secuencia SEQ_ALERTA	Identificador de la alerta
`id_emergencia`	NUMBER(10)	Sí	FK → EMERGENCIA	Emergencia relacionada (opcional)
`id_usuario`	NUMBER(10)	No	FK → USUARIO	Usuario destinatario
`tipo_alerta`	VARCHAR2(40)	No	Valores: CONGESTION, ACCIDENTE_EN_RUTA, CIERRE_VIA, RECALCULO_RUTA, CAMBIO_DISPONIBILIDAD, OTRO	Tipo de alerta
`mensaje`	VARCHAR2(300)	No	—	Contenido
`fecha_hora`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento
`leida`	CHAR(1)	No	Valores: S, N DEFAULT 'N'	S = leída, N = no leída

7.4.22 NOTIFICACION

Mensajes internos para los usuarios. Clase Java: `Notification`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_notificacion`	NUMBER(12)	No	PK Secuencia SEQ_NOTIFICACION	Identificador
`id_usuario`	NUMBER(10)	No	FK → USUARIO	Usuario destinatario
`tipo`	VARCHAR2(40)	No	—	Tipo de notificación (mayúsculas)
`titulo`	VARCHAR2(120)	No	—	Título
`mensaje`	VARCHAR2(400)	No	—	Mensaje
`fecha_hora`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento
`leida`	CHAR(1)	No	Valores: S, N DEFAULT 'N'	S = leída, N = no leída

Restricciones de la tabla:

- `ck_notificacion_tipo` (CHECK): `tipo = UPPER(tipo)`

7.4.23 AUDITORIA

Trazabilidad de operaciones críticas y de inicios de sesión. Clase Java: `AuditLog`.

Campo	Tipo Oracle	Nulo	Restricciones	Descripción
`id_auditoria`	NUMBER(12)	No	PK Secuencia SEQ_AUDITORIA	Identificador del registro
`id_usuario`	NUMBER(10)	Sí	FK → USUARIO	Usuario que realizó la operación (vacío si fue el sistema o un login fallido de un usuario inexistente)
`modulo`	VARCHAR2(60)	No	—	Módulo afectado (mayúsculas)
`operacion`	VARCHAR2(40)	No	—	LOGIN, INSERT, UPDATE, DESPACHO…
`registro_id`	NUMBER(12)	Sí	—	Identificador del registro afectado
`fecha_hora`	TIMESTAMP	No	DEFAULT SYSTIMESTAMP	Momento
`resultado`	VARCHAR2(20)	No	Valores: EXITO, FALLO	Éxito o fallo
`detalle`	VARCHAR2(500)	Sí	—	Información adicional

Restricciones de la tabla:

- `ck_auditoria_mayus` (CHECK): `modulo = UPPER(modulo) AND operacion = UPPER(operacion)`

7.5 Relaciones y cardinalidades

Entidad A	Entidad B	Cardinalidad	Explicación
ROL	USUARIO	1 : N	Un rol pertenece a muchos usuarios; cada usuario tiene un rol.
ROL	PERMISO	N : M (ROL_PERMISO)	Un rol tiene varios permisos y un permiso se asigna a varios roles.
ESTACION_EMERGENCIA	VEHICULO_EMERGENCIA	1 : N	Una estación administra varias unidades.
TIPO_VEHICULO	VEHICULO_EMERGENCIA	1 : N	Un tipo clasifica varias unidades.
HOSPITAL	SERVICIO_HOSPITAL	1 : N	Un hospital ofrece varios servicios.
TIPO_EMERGENCIA	EMERGENCIA	1 : N	Un tipo clasifica muchas emergencias.
USUARIO	EMERGENCIA	1 : N	Un operador registra muchas emergencias.
NODO_VIAL	TRAMO_VIAL	1 : N (dos veces)	Un nodo es origen o destino de varios tramos.
NODO_VIAL	SEMAFORO	1 : N	Una intersección puede tener varios semáforos.
TRAMO_VIAL	TRAFICO	1 : N	Un tramo acumula mediciones en el tiempo.
TRAMO_VIAL	INCIDENTE_VIAL	1 : N	Un tramo puede tener varios incidentes.

Entidad A	Entidad B	Cardinalidad	Explicación
EMERGENCIA	RUTA	1 : N	Una emergencia tiene una ruta principal y alternativas (y recálculos).
VEHICULO_EMERGENCIA	RUTA	1 : N	Una unidad recorre muchas rutas.
HOSPITAL	RUTA	0..1 : N	Un hospital es destino de muchas rutas de traslado.
RUTA	TRAMO_VIAL	N : M (RUTA_TRAMO)	Una ruta es una secuencia ordenada de tramos.
EMERGENCIA	VEHICULO_EMERGENCIA	N : M (ASIGNACION_EMERGENCIA)	Una emergencia puede requerir varias unidades; una unidad atiende muchas emergencias en el tiempo, pero solo una asignación abierta a la vez.
USUARIO	ASIGNACION_EMERGENCIA	1 : N	Un coordinador hace muchos despachos.
VEHICULO_EMERGENCIA	UBICACION_VEHICULO	1 : N	Se guardan muchas posiciones por unidad.
EMERGENCIA	ALERTA	0..1 : N	Una alerta puede referirse a una emergencia o ser general.
USUARIO	ALERTA / NOTIFICACION	1 : N	Un usuario recibe muchas alertas y notificaciones.
USUARIO	AUDITORIA	0..1 : N	Las operaciones de un usuario generan trazabilidad; el sistema también audita.

> **Decisión de diseño**
>
> Las rutas alternativas no necesitan una tabla propia: son filas adicionales de `RUTA` con la misma emergencia, diferenciadas por `tipo_ruta` y `es_recomendada`. Un recálculo crea una ruta nueva `ACTIVA` y deja la anterior `DESCARTADA`, lo que conserva el historial.

7.6 Catálogo de estados y valores

Severidad (un solo catálogo para EMERGENCIA e INCIDENTE_VIAL)

`BAJA` · `MEDIA` · `ALTA` · `CRITICA`

> Antes: EMERGENCIA tenía 4 niveles (con "crítica", con tilde) e INCIDENTE_VIAL solo 3.

EMERGENCIA.estado

Valor	Significado
`REPORTADA`	Registrada, sin unidad
`ASIGNADA`	Unidad seleccionada
`EN_RUTA`	Unidad desplazándose al sitio

Valor	Significado
EN_SITIO	Unidad en el lugar
TRASLADANDO	Unidad llevando al paciente al hospital
CERRADA	Atención finalizada (exige fecha_hora_cierre)

> Antes: el diccionario decía "en atención", que no existe en el Anexo A.

ASIGNACION_EMERGENCIA.estado

ASIGNADA · EN_RUTA · EN_SITIO · TRASLADANDO · FINALIZADA · CANCELADA

- Se agregan TRASLADANDO (para seguir la misma secuencia de la emergencia) y CANCELADA (cuando una unidad se reemplaza por otra, por ejemplo por una avería).
- Una unidad solo puede tener **una** asignación abierta (índice ux_asignacion_vehiculo_abierta).

VEHICULO_EMERGENCIA.estado_operativo

DISPONIBLE · OCUPADO · MANTENIMIENTO · FUERA_SERVICIO

> Antes: "Disponible, en ruta, mantenimiento, fuera de servicio". Se cambió "en ruta" por OCUPADO porque la unidad también está ocupada cuando está en el sitio o trasladando. **La fase de la misión se consulta en `ASIGNACION_EMERGENCIA.estado`**; así no se repite el mismo dato en dos tablas.

TRAMO_VIAL.estado

ABIERTO · CERRADO · RESTRINGIDO

INCIDENTE_VIAL

Columna	Valores
tipo_incidente	ACCIDENTE · OBRA · CIERRE · MANIFESTACION · OTRO
estado	PENDIENTE · ACTIVO · CERRADO · RECHAZADO
origen	OPERADOR · QR_CIUDADANO · SIMULACION
bloquea_tramo	S (el motor lo trata como cerrado) · N (solo penaliza)

- **El estado por defecto es `PENDIENTE`.** Un incidente solo afecta las rutas cuando alguien lo pasa explícitamente a ACTIVO. Así un reporte ciudadano falso nunca cierra una calle por error.
- RECHAZADO sirve para descartar un reporte que no se validó.

TRAFICO

Columna	Valores
nivel	FLUIDO · MODERADO · CONGESTIONADO
fuente	MANUAL · SIMULACION · SENSOR · API_EXTERNA

Columna	Valores
factor_congestion	≥ 1.00 (1.00 = flujo libre; 2.00 = el doble de lento)

UBICACION_VEHICULO.fuente (columna nueva)

SIMULACION · GPS

> Se agrega para cumplir el requisito de **no presentar datos simulados como reales**.

RUTA

Columna	Valores
tipo_ruta	PRINCIPAL · ALTERNATIVA
estado	PROPUESTA · ACTIVA · FINALIZADA · DESCARTADA
es_recomendada	S · N

- id_hospital_destino vacío = trayecto hacia la emergencia; con valor = traslado al hospital.
- Una emergencia y una unidad tienen como máximo una ruta ACTIVA (índice ux_ruta_activa).

Otros catálogos

Tabla.columna	Valores
SEMAFORO.estado_actual	VERDE · AMARILLO · ROJO · FUERA_SERVICIO
NODO_VIAL.tipo_nodo	INTERSECCION · GLORIETA · PUENTE · ACCESO · PUNTO_VIA · OTRO
HOSPITAL.nivel	I · II · III · IV
ALERTA.tipo_alerta	CONGESTION · ACCIDENTE_EN_RUTA · CIERRE_VIA · RECALCULO_RUTA · CAMBIO_DISPONIBILIDAD · OTRO
AUDITORIA.resultado	EXITO · FALLO
Prioridad (TIPO_EMERGENCIA, TIPO_VEHICULO)	1 = más baja … 10 = más alta

7.7 Secuencias e índices

21 secuencias, una por tabla con llave simple (las tablas ROL_PERMISO y RUTA_TRAMO tienen llave compuesta): SEQ_ROL, SEQ_PERMISO, SEQ_USUARIO, SEQ_ESTACION_EMERGENCIA, SEQ_TIPO_VEHICULO, SEQ_VEHICULO_EMERGENCIA, SEQ_HOSPITAL, SEQ_SERVICIO_HOSPITAL, SEQ_TIPO_EMERGENCIA, SEQ_EMERGENCIA, SEQ_NODO_VIAL, SEQ_TRAMO_VIAL, SEQ_SEMAFORO, SEQ_TRAFICO, SEQ_INCIDENTE_VIAL, SEQ_RUTA, SEQ_ASIGNACION_EMERGENCIA, SEQ_UBICACION_VEHICULO, SEQ_ALERTA, SEQ_NOTIFICACION, SEQ_AUDITORIA.

24 índices propios (Oracle no indexa las llaves foráneas automáticamente):

Índice	Tabla	Columnas o expresión
ix_usuario_rol	USUARIO	id_rol
ix_vehiculo_estacion	VEHICULO_EMERGENCIA	id_estacion
ix_vehiculo_estado	VEHICULO_EMERGENCIA	estado_operativo
ix_emergencia_estado	EMERGENCIA	estado
ix_emergencia_fecha	EMERGENCIA	fecha_hora_reporte
ix_emergencia_tipo	EMERGENCIA	id_tipo_emergencia
ix_tramo_origen	TRAMO_VIAL	id_nodo_origen
ix_tramo_destino	TRAMO_VIAL	id_nodo_destino
ix_semaforo_nodo	SEMAFORO	id_nodo
ix_trafico_tramo_fecha	TRAFICO	id_tramo, fecha_hora
ix_incidente_tramo	INCIDENTE_VIAL	id_tramo
ix_incidente_estado	INCIDENTE_VIAL	estado
ix_ruta_emergencia	RUTA	id_emergencia
ix_ruta_vehiculo	RUTA	id_vehiculo
ix_ruta_tramo_tramo	RUTA_TRAMO	id_tramo
ix_asignacion_emergencia	ASIGNACION_EMERGENCIA	id_emergencia
ix_asignacion_vehiculo	ASIGNACION_EMERGENCIA	id_vehiculo
ix_ubicacion_vehiculo_fecha	UBICACION_VEHICULO	id_vehiculo, fecha_hora
ix_alerta_usuario_leida	ALERTA	id_usuario, leida
ix_notificacion_usuario	NOTIFICACION	id_usuario, leida
ix_auditoria_fecha	AUDITORIA	fecha_hora
ix_auditoria_usuario	AUDITORIA	id_usuario
ux_asignacion_vehiculo_abierta	ASIGNACION_EMERGENCIA	CASE WHEN estado NOT IN ('FINALIZADA', 'CANCELADA') THEN id_vehiculo END
ux_ruta_activa	RUTA	CASE WHEN estado = 'ACTIVA' THEN id_emergencia END, CASE WHEN estado = 'ACTIVA' THEN id_vehiculo END

7.8 Reglas que garantiza la base de datos

- **Una unidad no puede tener dos asignaciones abiertas:** índice único parcial `ux_asignacion_vehiculo_abierta` (solo indexa las asignaciones que no están `FINALIZADA` ni `CANCELADA`).
- **Una emergencia y una unidad tienen como máximo una ruta activa:** índice único parcial `ux_ruta_activa`.
- **El tiempo base de un tramo nunca queda desactualizado:** `tiempo_base_seg` es una columna virtual calculada con la distancia y la velocidad.

- **Una emergencia cerrada siempre tiene fecha de cierre**, y una abierta nunca la tiene (`ck_emergencia_cierre`).
- **Un tramo no puede empezar y terminar en el mismo nodo** (`ck_tramo_nodos`).
- **Los estados solo aceptan los valores del catálogo**, en mayúsculas (restricciones CHECK). La prueba `EnumSqlContractTest` verifica que coincidan con los enums de Java.
- **Coordenadas válidas:** latitud entre -90 y 90, longitud entre -180 y 180.
- **Un incidente nuevo queda `PENDIENTE`** por defecto y no afecta las rutas hasta que se activa.

7.9 Cambios respecto al diccionario original

#	Tabla	Cambio	Motivo
1	TRAMO_VIAL	Se elimina `sentido`. **Cada fila es un solo sentido de circulación**; una vía de doble sentido son dos filas	El grafo usa aristas dirigidas. Con una sola fila para los dos sentidos, `RUTA_TRAMO` no podría saber en qué dirección se recorrió el tramo
2	TRAMO_VIAL	`tiempo_base_seg` pasa a ser columna **virtual** (`distancia_m / (velocidad_max_kmh / 3.6)`) y `velocidad_max_kmh` pasa a ser obligatoria	El tiempo base nunca queda desactualizado respecto a la distancia o la velocidad
3	NODO_VIAL, TRAMO_VIAL	Columnas opcionales `osm_id` y `osm_way_id`	Saber de qué elemento de OpenStreetMap salió cada dato y poder recargar la red sin duplicados
4	RUTA_TRAMO	Clave primaria `(id_ruta, orden_tramo)` en lugar de `(id_ruta, id_tramo)`	Lo que identifica cada fila es su posición en la secuencia
5	EMERGENCIA	Columna nueva `fecha_hora_cierre` (obligatoria cuando el estado es `CERRADA`)	Calcular el tiempo total de atención en el dashboard
6	INCIDENTE_VIAL	Columnas nuevas `origen` y `bloquea_tramo`; estados `PENDIENTE` y `RECHAZADO`	Módulo QR y regla explícita de cierre o penalización
7	UBICACION_VEHICULO	Columna nueva `fuente`	Distinguir los datos simulados de los reales
8	ALERTA	`id_emergencia` pasa a ser opcional	Existen alertas generales (cambio de disponibilidad, congestión)
9	AUDITORIA	`id_usuario` pasa a ser opcional	Registrar inicios de sesión fallidos y acciones del simulador
10	PERMISO, SERVICIO_HOSPITAL	Restricciones únicas `(modulo, accion)` y `(id_hospital, nombre_servicio)`	Evitar registros duplicados

7.10 Datos iniciales

Roles

Rol	Descripción
`ADMINISTRADOR`	Administra usuarios, catálogos y configuracion
`OPERADOR_CENTRAL`	Registra emergencias y consulta rutas

Rol	Descripción
`COORDINADOR`	Despacha unidades y supervisa la operación
`BOMBERO_PARAMEDICO`	Personal en la unidad; consulta su misión y su ruta

Permisos por rol

Cada módulo tiene dos acciones: **CONSULTAR** (ver) y **GESTIONAR** (crear, editar y cambiar estados). Total: 20 permisos y 53 asignaciones.

Módulo	Acción	Administrador	Operador	Coordinador	Bombero / Paramédico
USUARIOS	CONSULTAR	✓			
USUARIOS	GESTIONAR	✓			
EMERGENCIAS	CONSULTAR	✓	✓	✓	✓
EMERGENCIAS	GESTIONAR	✓	✓	✓	
UNIDADES	CONSULTAR	✓	✓	✓	
UNIDADES	GESTIONAR	✓		✓	
HOSPITALES	CONSULTAR	✓	✓	✓	✓
HOSPITALES	GESTIONAR	✓		✓	
RED_VIAL	CONSULTAR	✓	✓	✓	
RED_VIAL	GESTIONAR	✓			
RUTAS	CONSULTAR	✓	✓	✓	✓
RUTAS	GESTIONAR	✓	✓	✓	
TRAFICO	CONSULTAR	✓	✓	✓	
TRAFICO	GESTIONAR	✓		✓	
INCIDENTES	CONSULTAR	✓	✓	✓	
INCIDENTES	GESTIONAR	✓	✓	✓	
DESPACHO	CONSULTAR	✓	✓	✓	✓
DESPACHO	GESTIONAR	✓		✓	
REPORTES	CONSULTAR	✓	✓	✓	
AUDITORIA	CONSULTAR	✓		✓	

Tipos de vehículo y de emergencia

Tabla	Nombre	Prioridad	Descripción
TIPO_VEHICULO	`AMBULANCIA`	8	Unidad de atención médica y traslado
TIPO_VEHICULO	`BOMBERO`	8	Vehículo del cuerpo de bomberos
TIPO_EMERGENCIA	`EMERGENCIA_MEDICA`	9	Urgencia médica sin accidente de tránsito
TIPO_EMERGENCIA	`INCENDIO`	9	Incendio estructural, vehicular o forestal

Tabla	Nombre	Prioridad	Descripción
TIPO_EMERGENCIA	`ACCIDENTE_VIAL`	8	Accidente de tránsito con posibles lesionados
TIPO_EMERGENCIA	`RESCATE`	7	Rescate de personas atrapadas o en riesgo

Las prioridades (1 = más baja, 10 = más alta) son **provisionales** y el equipo debe justificarlas. No se cargan usuarios, hospitales ni estaciones de ejemplo: el primer usuario se crea desde Java (el hash lo genera el backend) y los hospitales y estaciones se cargarán con datos reales verificados de Valledupar.

7.11 Scripts y verificación

Script	Se ejecuta como	Propósito
`00_crear_usuario.sql`	SYSTEM	Crea el usuario `RUTAVIDA` en `XEPDB1` (pide la contraseña sin mostrarla)
`01_esquema.sql`	RUTAVIDA	23 tablas, 21 secuencias, 24 índices, restricciones y comentarios
`02_datos_iniciales.sql`	RUTAVIDA	Roles, permisos, asignación de permisos y tipos
`98_verificar.sql`	RUTAVIDA	Comprueba el esquema y prueba las reglas (todo se deshace con ROLLBACK)
`99_eliminar_esquema.sql`	RUTAVIDA	Borra todo para volver a crearlo desde cero

Resultado de la ejecución en Oracle XE 18c (14/10/2026)

Comprobación	Esperado	Obtenido
Tablas	23	**23 · SI**
Secuencias	21	**21 · SI**
Objetos inválidos	0	**0 · SI**
Restricciones	—	23 PK · 26 FK · 12 UNIQUE · 205 CHECK (incluye NOT NULL)
Datos iniciales	4 roles · 20 permisos · 53 asignaciones · 2 + 4 tipos	**Todos SI**
Columna virtual (1000 m a 36 km/h)	100 s	**100 · SI**
Estado en minúsculas / tramo a sí mismo	Rechazados (ORA-02290)	**Rechazados**
Incidente nuevo	PENDIENTE	**PENDIENTE · SI**
Conexión desde Java (`DatabaseCheck`)	23 de 23	**23 de 23**
Índices propios	24	La primera ejecución creó 22 (ver nota). Se crearon a mano los dos faltantes; falta confirmar `INDICES 24 SI` con `98_verificar.sql`

> **Lección aprendida**
>
> En SQL*Plus una sentencia termina solo si el `;` es lo último de la línea. Un comentario después del `;` hizo

> que dos `CREATE INDEX` se enviaran juntos (error ORA-02158). El script ya está corregido y la prueba `SqlScriptStyleTest` impide que vuelva a pasar.

7.12 Tráfico e incidentes en el motor de rutas

La base de datos guarda el **historial**; el estado vigente de cada tramo vive en memoria, en el grafo. Para que el simulador de tráfico no "reabra" un tramo cerrado por un incidente, el estado que usa el motor se calcula **combinando dos fuentes**:

El simulador solo modifica la congestión y los semáforos; **no cierra tramos**. Los cierres vienen de incidentes (creados por el operador o por un escenario de demostración). Este acuerdo debe reflejarse en el diseño de `EdgeState` y `TrafficUpdater` (Kendrick).

Persistencia por lotes: `TRAFICO` y `UBICACION_VEHICULO` no se escriben en cada tick del simulador. Se guardan cada N ticks o solo cuando el cambio es relevante (valor de N por definir).

7.13 PL/SQL planificado

Tipo	Objetos	Finalidad	Estado
Procedimientos	`sp_asignar_unidad`, `sp_cerrar_emergencia`	Procesos transaccionales y cálculo de tiempos de respuesta	Pendiente
Triggers	Auditoría en `EMERGENCIA`, `ASIGNACION_EMERGENCIA`, `INCIDENTE_VIAL`	Trazabilidad automática de cambios	Pendiente
Vistas	`vw_emergencias_activas`, `vw_unidades_disponibles`, `vw_hospitales_operativos`	Consultas del dashboard	Pendiente
Funciones	`fn_tiempo_respuesta`, `fn_unidades_disponibles`	Cálculos reutilizables para reportes	Pendiente

8. Motor de rutas y tiempo real

Estado: **diseñado**. El código de referencia está en la Parte IV de la versión 1.0, adaptado en el diagrama de clases (sección 6). Lo implementa Kendrick en el Sprint 1 (tareas K1 y K2).

8.1 Modelo de grafo

- La topología (nodos y tramos) se carga **una vez** desde `NODO_VIAL` y `TRAMO_VIAL` y queda en memoria (`RoadGraph`). El motor **no consulta la base de datos** durante el cálculo.
- Los nodos y tramos usan índices internos 0..n-1 para ser rápidos, y guardan su `dbId` para volver a Oracle al guardar `RUTA_TRAMO`.
- El estado de cada tramo (`EdgeState`: congestión, cerrado, espera de semáforo, factor de incidente) es un objeto inmutable que se **reemplaza completo** en cada actualización. Así el cálculo nunca lee valores a medias.
- Solo `RoadConditionService` modifica el grafo. Combina el tráfico del simulador con los incidentes activos.

8.2 Función de costo

> **Costo de un tramo, en segundos**
>
> **costo = tiempo de recorrido + espera por semáforo**, multiplicado por el factor de incidente.
>
> tiempo de recorrido = distancia ÷ velocidad efectiva.
>
> velocidad efectiva = mín(velocidad máxima del tramo, velocidad máxima del vehículo) ÷ congestión efectiva.
>
> congestión efectiva = 1 + (congestión − 1) × (1 − alivio de congestión del vehículo).
>
> espera por semáforo = espera esperada × (1 − alivio de semáforo del vehículo).
>
> Si el tramo está cerrado (por estado o por un incidente que lo bloquea), el costo es **infinito** y el tramo se evita.

Perfil	Alivio de congestión	Alivio de semáforo	Velocidad máxima
Ambulancia	0,35	0,60	90 km/h
Bomberos	0,25	0,50	80 km/h

*Estos valores son **supuestos a calibrar**: representan cuánto esquiva una unidad de emergencia la congestión y los semáforos. Deben ser configurables y justificarse como hipótesis, no como datos medidos.*

8.3 A* y Dijkstra

- A* usa como heurística la distancia en línea recta (haversine) dividida por la velocidad máxima posible. Es **admisible**: nunca sobreestima, porque ningún tramo se puede recorrer más rápido. Por eso A* encuentra la misma ruta óptima que Dijkstra, explorando menos nodos.
- Con la heurística desactivada, el mismo código se comporta como Dijkstra. Así se pueden comparar en la sustentación: mismo costo, distinta cantidad de nodos explorados y distinto tiempo de cálculo.

- `DistanceOnlyCost` calcula la ruta más corta en distancia, para mostrar que no es la más rápida cuando hay congestión.

8.4 Rutas alternativas

Se usa el **método de penalización**, más simple que el algoritmo de Yen y suficiente para el alcance: se calcula la mejor ruta, se penalizan sus tramos en una copia local del costo (sin tocar el grafo real) y se vuelve a buscar. Se descartan las rutas que comparten más del 80 % de los tramos con otra ya elegida o que tardan más de 1,5 veces la mejor (regla RN-08). El tiempo que se reporta se recalcula sin las penalizaciones artificiales.

8.5 Recálculo de rutas activas

Ante un cambio de tráfico o un incidente activado, `RouteMonitor` recalcula las rutas activas afectadas desde la **posición actual** de la unidad. Solo reemplaza la ruta si la mejora supera el 15 % o los 30 segundos, o si la ruta actual quedó bloqueada (regla RN-09). Así la ruta no "parpadea" por cambios mínimos.

8.6 Asignación de unidades y de hospital

- **Una emergencia:** se elige la unidad disponible con menor tiempo estimado de llegada (ETA), calculado con A*.
- **Varias emergencias a la vez:** se arma la matriz de ETA (emergencias × unidades) y se resuelve el problema de asignación con el **algoritmo húngaro**, de complejidad $O(n^3)$. Es el modelo clásico de Investigación de Operaciones.
- **Hospital de traslado:** costo = ETA del traslado + penalización por baja disponibilidad, descartando los hospitales sin el servicio requerido (regla RN-10).

8.7 Fase 1 (simulación) frente a fase 2 (datos reales)

Componente	Fase 1 — simulación	Fase 2 — datos reales
Red vial	Extracto de OpenStreetMap de Valledupar en `NODO_VIAL` y `TRAMO_VIAL`	El mismo grafo, actualizado periódicamente
Tráfico	`SimulatedTrafficProvider`: hora pico, variación aleatoria con semilla fija y escenarios de demostración	Otra implementación de `TrafficProvider` con una fuente externa (verificar cobertura y precio vigentes antes de prometerla)
Semáforos	Ciclos simulados con espera esperada	Datos de la autoridad de movilidad, si existen; si no, espera promedio calibrada
Posición de las unidades	`VehicleMovementSimulator` mueve la unidad por su ruta	Otra implementación de `VehiclePositionSource` con GPS
Base de datos	Oracle XE local	Oracle centralizado (en la nube o en un servidor)
Seguridad	Login con hash y roles	Además: HTTPS, límite de solicitudes y auditoría completa

Lo único que cambia entre fases es **de dónde vienen los datos**. Por eso esas fuentes están detrás de interfaces (`TrafficProvider`, `VehiclePositionSource`) desde el primer día: GRASP *Protected Variations*.

8.8 Métricas para la sustentación

- Tiempo de cálculo de una ruta, en milisegundos.
- Nodos explorados por A* frente a Dijkstra.
- ETA con y sin penalización de tráfico.
- Porcentaje de mejora de la ruta recomendada frente a la ruta de menor distancia.
- Tiempo ganado en cada recálculo.
- Tiempo promedio de respuesta (asignación → llegada) en el dashboard.

9. Estado de la implementación y pruebas

Esta sección distingue con precisión lo que **funciona**, lo que está **programado**, lo que está **diseñado** y lo que está **pendiente**.

9.1 Repositorio

Carpeta o archivo	Contenido
`backend/`	Proyecto Java 21 + Maven, paquete `co.edu.unicesar.rutavida`
`database/`	Scripts SQL en orden numérico (00, 01, 02, 98, 99)
`docs/`	Casos de uso, diagrama de clases, arquitectura, convenciones de base de datos y plan de trabajo
`README.md`	Puesta en marcha y estado del proyecto
`CONTRIBUTING.md`	Guía de trabajo en equipo con Git

GitHub: `afzabaleta/RutaVida`, rama `develop`.

9.2 Qué está hecho

Componente	Estado	Evidencia
Base de datos (23 tablas, secuencias, índices, restricciones, datos iniciales)	**Funciona**	Ejecutada en Oracle XE 18c; `98_verificar.sql` (sección 7.11)
Conexión Java ↔ Oracle	**Funciona**	`DatabaseCheck`: "23 de 23"
15 paquetes con su responsabilidad documentada	**Programado**	`package-info.java` en cada paquete
19 enums de estados y catálogos	**Programado y probado**	`EnumSqlContractTest`, `StatusTransitionTest`
Transacciones (`TransactionManager`, `JdbcTransactionManager`)	**Programado y probado**	`JdbcTransactionManagerTest` con conexiones falsas
Eventos de tiempo real (`EventBus`, `InMemoryEventBus`)	**Programado y probado**	`InMemoryEventBusTest`
Excepciones y `AppContext`	**Programado**	Compila; usado por `DatabaseCheck`
Entidades, DAOs, servicios, controladores	Diseñado	Diagrama de clases; Sprint 1
Motor de rutas y simulación	Diseñado	Sección 8; Sprint 1 (K1, K2)
Vista	Pendiente	Depende de la decisión Swing o web
PL/SQL (procedimientos, triggers, vistas)	Pendiente	Sección 7.13

9.3 Pruebas automáticas

Clase de prueba	Pruebas	Qué verifica
`ArchitectureTest`	9	Que ninguna capa importe lo que no le corresponde
`EnumSqlContractTest`	3	Que los 19 enums coincidan con los CHECK de Oracle

Clase de prueba	Pruebas	Qué verifica
`StatusTransitionTest`	6	Las transiciones de estado permitidas y prohibidas
`JdbcTransactionManagerTest`	6	Commit, rollback, conexión compartida y errores de JDBC
`InMemoryEventBusTest`	4	Entrega de eventos por tipo, cancelación y aislamiento de errores
`DatabaseConnectionTest`	6	Configuración de la conexión y que la contraseña nunca se muestre
`SqlScriptStyleTest`	1	Que ningún script tenga un comentario después del `;`
`ClassDiagramSyncTest`	2	Que toda clase del código esté en el diagrama de clases
Total	**37**	

Resultado: 35 pruebas pasaron en IntelliJ en el equipo de Andrés el 14/10/2026. Las 2 de `ClassDiagramSyncTest` se agregaron después: pasan en el entorno de verificación y falta correrlas en IntelliJ.

Además se comprobó que las pruebas **detectan errores de verdad**: se rompieron reglas a propósito en una copia (un servicio que importa JDBC, un estado distinto en el SQL, un comentario después del `;`, una clase fuera del diagrama) y cada prueba falló como debía.

10. Organización del equipo

10.1 Roles

Integrante	Responsabilidad principal	Paquetes
Andrés (líder)	Base de datos Oracle y PL/SQL, DAO, integración, revisión de Pull Requests y documento final	`dao`, `dao.jdbc`, `app`, `event`, `exception`, `database/`
Kendrick	Motor de rutas, tráfico, simulación y servicios de despacho, rutas y seguimiento	`graph`, `traffic`, `simulation`, `service` (rutas, despacho, seguimiento)
Sherly	Modelo, utilidades, controladores, servicios de usuarios, hospitales, alertas y auditoría, y pruebas	`model`, `dto`, `util`, `controller`, `service` (usuarios, hospitales, alertas)
Diego	Vista, versión móvil, mockups y datos reales de Valledupar	`view` y la carpeta de la vista web si se elige

10.2 Convenciones de idioma

Qué	Idioma	Ejemplo
Documentación: README, informes, manuales, diagramas	Español	`docs/analysis.md`
Control de versiones: commits, ramas, Pull Requests	Español	`funcionalidad: agrega A* con heuristica haversine`
Código fuente: clases, métodos, variables, comentarios	Inglés	`class EmergencyService`, `calculateOptions()`
Lo que ve el usuario: pantallas, botones, mensajes	Español	"Emergencia registrada correctamente"
Tablas y columnas de Oracle	Español (pendiente de confirmar)	`TRAMO_VIAL` ↔ `RoadSegment`

10.3 Flujo de trabajo con Git

Rama	Para qué	Quién escribe
`main`	Versión estable que se presenta	Solo por Pull Request desde `develop`, cuando el equipo lo decide
`develop`	Integración del trabajo de todos	Solo por Pull Request
`funcionalidad/<nombre>`	Una funcionalidad nueva	Su responsable
`correccion/<nombre>`	Arreglar un error	Quien lo arregla
`documentacion/<nombre>`	Documentos	Quien los escribe
`diseno/<nombre>`	Mockups y diseño	Diego
`datos/<nombre>`	Scripts con datos reales	Quien los recopila

1. Partir de `develop` actualizado: `git checkout develop`, `git pull`, `git checkout -b funcionalidad/<nombre>`.

2. Hacer commits pequeños con el formato `tipo: mensaje` en español.
3. Subir la rama: `git push -u origin funcionalidad/<nombre>`.
4. Abrir un Pull Request hacia `develop`, con la descripción de qué cambia y cómo se prueba.
5. Un compañero revisa, corre las pruebas y aprueba; luego se integra y se borra la rama.

Nadie sube cambios directamente a `main` ni a `develop`. Antes de cada Pull Request todas las pruebas deben pasar.

10.4 Plan del Sprint 1

Objetivo: iniciar sesión funcionando de punta a punta (CU-01) y el motor de rutas probado con un grafo de prueba, mientras se preparan los datos reales y el diseño de las pantallas.

Andrés — base de datos, DAO e integración

#	Tarea	Rama	Depende de
A1	Base común de los DAO: `RowMapper`, `AbstractJdbcDAO` y la excepción `AccessDeniedException`	`funcionalidad/base-dao`	—
A2	`UserDAO`, `RoleDAO`, `AuditDAO` (interfaces + `JdbcXxxDAO`) con pruebas de integración contra Oracle	`funcionalidad/dao-seguridad`	A1, S1
A3	Conectar el login en `AppContext` y una herramienta `CreateAdminUser` que crea el primer administrador con la contraseña en hash	`funcionalidad/integracion-login`	A2, S2
A4	Proteger el repositorio: colaboradores, rama por defecto `develop` y reglas de rama (ver `CONTRIBUTING.md`, sección 5)	—	—

Terminado cuando:

- A1 y A2: `mvn test` pasa. Las pruebas de los DAO insertan, leen y actualizan en el Oracle local y **se saltan solas** si no existe `db.properties`, para que no fallen en otros equipos.
- A3: el administrador creado con `CreateAdminUser` inicia sesión y queda un registro `LOGIN / EXITO` en `AUDITORIA`.

Sherly — Model, Util, servicios de usuarios, Controller y pruebas

#	Tarea	Rama	Depende de
S1	Entidades del modelo: `GeoPoint`, `Locatable`, `User`, `Role`, `Permission`, `UserSession`, `Emergency`, `EmergencyType`, `EmergencyVehicle`, `VehicleType`, `EmergencyStation`, `Hospital`	`funcionalidad/modelo-base`	—
S2	`PasswordHasher` + `Pbkdf2PasswordHasher` (PBKDF2 con sal, viene en Java), `AuditService`, `AuthService`, `SessionContext`, `OperationResult`, `LoginController` y `Messages`	`funcionalidad/login`	S1

Terminado cuando:

- S1: las entidades validan sus datos en el constructor (por ejemplo, `GeoPoint` rechaza una latitud fuera de -90 a 90) y tienen pruebas JUnit.

- S2: `AuthService` tiene pruebas **con DAOs falsos**, sin Oracle, que cubren: login correcto, contraseña incorrecta, usuario inexistente (el mismo mensaje en ambos casos), usuario inactivo y auditoría registrada en cada caso. El hash de la misma contraseña da resultados distintos (por la sal) y `matches` funciona.

Orden recomendado: S1 primero y rápido, porque A2 depende de las entidades. Mientras Andrés hace A2, Sherly avanza S2 usando las interfaces DAO del diagrama.

Kendrick — motor de rutas y tráfico

#	Tarea	Rama	Depende de
K1	Paquete `graph`: `Node`, `Edge`, `EdgeState`, `VehicleProfile`, `RoadGraph`, `CostFunction`, `EmergencyCost`, `DistanceOnlyCost`, `RouteResult`, `AStarRouter`, `AlternativeRouteFinder`	`funcionalidad/motor-rutas`	—
K2	Paquete `traffic`: `TrafficProvider`, `EdgeUpdate`, `SimulatedTrafficProvider` con semilla fija (resultados repetibles)	`funcionalidad/trafico-simulado`	K1

El documento consolidado (Parte IV) tiene código de referencia de K1. Hay que adaptarlo a los cambios de `docs/class-diagram.md`, sección 6: `dbId` en `Node` y `Edge`, `incidentFactor` en `EdgeState`, `EdgeUpdate` sin `closed` y el nombre `AlternativeRouteFinder`.

Terminado cuando: hay pruebas JUnit con un grafo de 6 nodos hecho a mano que demuestran que:

1. A* y Dijkstra (`useHeuristic = false`) dan el mismo costo.
2. Un tramo cerrado se evita.
3. Con congestión alta, la ruta más rápida no es la más corta (`EmergencyCost` contra `DistanceOnlyCost`).
4. Las alternativas cumplen la regla RN-08 (máximo 80 % de tramos en común y máximo 1,5 veces el tiempo de la mejor).
5. `SimulatedTrafficProvider` con la misma semilla produce los mismos cambios y nunca cierra tramos.

Además, una prueba de rendimiento que mida en milisegundos el tiempo de A* contra Dijkstra y la cantidad de nodos explorados por cada uno (métrica para la sustentación).

Diego — vista y datos reales

#	Tarea	Rama	Depende de
D1	Mockups de 5 pantallas: inicio de sesión, pantalla principal (mapa + tarjetas de indicadores + panel de emergencias), registrar emergencia, despacho con las 3 rutas y su explicación, y control de la simulación. Interfaz oscura con detalles rojos y azules; logo arriba a la izquierda, usuario y rol arriba a la derecha	`diseno/mockups`	—
D2	Datos reales de Valledupar: hospitales y clínicas con urgencias, estaciones de bomberos y bases de ambulancias, con dirección, coordenadas y **fuente de cada dato**. Entregarlos como `database/03_datos_valledupar.sql`	`datos/hospitales-estaciones`	—

La tecnología de la vista (Swing o web) **depende del profesor**. Mientras tanto, Diego trabaja en tareas que sirven en cualquier caso:

Terminado cuando:

- D1: las imágenes de los mockups están en `docs/mockups/` y aparecen en `docs/mockups.md`, con una línea por pantalla que indique qué caso de uso cubre.
- D2: el script se ejecuta sin errores en Oracle y `98_verificar.sql` sigue saliendo bien. Cada fila tiene un comentario con su fuente. Ningún dato inventado: si no se encuentra la coordenada de un lugar, ese lugar no se incluye. Andrés revisa este PR porque toca `database/`.

Cuando el profesor decida, Diego empieza la vista real (`funcionalidad/vista-login`, `funcionalidad/vista-mapa`).

Calendario

Días	Andrés	Sherly	Kendrick	Diego
1–2	A1, A4	S1	K1 (grafo y A*)	D1
3–4	A2	S2	K1 (alternativas y pruebas)	D1, D2
5	A3 + revisión de PRs	S2 + pruebas	K2	D2
Cierre	Demostración interna: login real + motor de rutas en pruebas			

Sprint 2

Red vial real (OpenStreetMap → `NODO_VIAL` / `TRAMO_VIAL` → `GraphLoader`), registrar emergencia (CU-06), calcular rutas sobre el mapa real (CU-08) y despachar unidad (CU-09).

11. Riesgos y recomendaciones

Riesgo	Impacto	Recomendación
Cartografía y geodatos	Construir el mapa desde cero es inviable	Usar un extracto acotado de OpenStreetMap de una zona de Valledupar (cientos a pocos miles de nodos), cargado una sola vez
Tráfico en tiempo real	Las fuentes externas pueden no cubrir Valledupar o tener costo	Fase 1 con simulador; la fuente real queda como fase 2, sin prometerla antes de verificar cobertura y precio
Base de datos local	Cada integrante tiene su propia base; los datos no se comparten	Toda estructura y dato inicial en scripts versionados; informar al profesor el cambio de nube a local
Decisión de la vista	Diego no puede empezar la interfaz real	Preguntar al profesor esta semana; mientras tanto, mockups y datos reales
Datos reales de hospitales y estaciones	Datos inventados restan credibilidad	Cada dato con su fuente; si no hay coordenadas verificadas, no se incluye
Algoritmo	Errores difíciles de ver en el mapa	Probar primero con grafos pequeños de resultado conocido (JUnit) antes de usar el mapa real
Alcance	Muchos módulos para el tiempo disponible	Priorizar el núcleo: login → emergencia → rutas → despacho → simulación → recálculo → cierre. El QR va al final
Integración entre integrantes	Clases duplicadas o firmas incompatibles	Contratos en el diagrama de clases, Pull Requests revisados y pruebas de arquitectura

12. Decisiones pendientes

Decisión	Responsable	Afecta a
Vista en Swing o web	Andrés pregunta al profesor	Diego y toda la capa de vista
Oracle local en lugar de la nube	Andrés informa al profesor	Todos
Nombres de tablas en español	Andrés pregunta al profesor	Base de datos y DAO
Zona de Valledupar a cargar de OpenStreetMap	Andrés y Kendrick	Sprint 2 (red vial real)
Justificación de las prioridades de tipos de emergencia y de vehículo	Equipo	Despacho y reportes
Calibración de los perfiles de vehículo y de los umbrales (RN-08, RN-09)	Kendrick	Motor de rutas
Aplicación móvil: web adaptable o nativa	Diego	Vista

Anexo A. Documentos del repositorio

Archivo	Contenido
`docs/analysis.md`	Casos de uso completos con diagramas Mermaid

Archivo	Contenido
`docs/class-diagram.md`	Diagrama de clases en Mermaid por capas
`docs/arquitectura-software.md`	Arquitectura, paquetes, GRASP y SOLID
`docs/convenciones-base-datos.md`	Convenciones, estados y cambios del modelo de datos
`docs/plan-de-trabajo.md`	Tareas del Sprint 1 por integrante
`CONTRIBUTING.md`	Guía de Git y de trabajo en equipo
`database/*.sql`	Scripts de la base de datos

Anexo B. Glosario

Término	Significado
A*	Algoritmo de caminos mínimos que usa una estimación (heurística) de lo que falta para llegar; explora menos nodos que Dijkstra
Dijkstra	Algoritmo clásico de caminos mínimos en grafos con pesos no negativos
ETA	Tiempo estimado de llegada
DAO	Objeto de acceso a datos: la única capa que ejecuta SQL
DTO	Objeto de transferencia de datos entre el controlador y la vista
Transacción	Grupo de operaciones que se guardan todas o ninguna
PBKDF2	Algoritmo de hash de contraseñas con sal e iteraciones; incluido en Java
OpenStreetMap (OSM)	Mapa colaborativo de datos abiertos de donde se extrae la red vial
Grafo dirigido	Grafo cuyas aristas tienen sentido; aquí, un tramo por sentido de circulación
Composition root	Lugar único donde se crean los objetos y se conectan sus dependencias (`AppContext`)
Tick	Cada paso periódico del simulador (por defecto, 5 segundos)